Guía de desarrollo en equipo: Sistema Airlines-poo en Java (POO)

Oct 10, 2026 · @Moraj Tech

1. Presentación y cómo usar esta guía

Esta guía es el punto de partida para que cinco estudiantes construyan, cada uno, un módulo de un sistema de reservas aéreas en Java aplicando Programación Orientada a Objetos (POO). El sistema se basa en el modelo de datos de Airlines-poo, pero en esta etapa **no usa base de datos**: los datos viven en memoria. En las siguientes clases reemplazaremos esa memoria por JDBC o JPA.

Esta guía no contiene código Java. Describe qué clases crear, qué atributos y comportamientos darles, qué reglas validar y qué conceptos de clase practicar. El código lo escribes tú: ese es el ejercicio.

A quién va dirigida

A estudiantes de segundo nivel de desarrollo de software que ya vieron variables, estructuras de control, métodos y los primeros conceptos de POO (clases, objetos, encapsulamiento). Los temas que van más allá de lo visto en el curso (por ejemplo excepciones propias, fechas y horas con java.time, UUID o Comparator) aparecen marcados con 🔎: para esos, consulta la documentación oficial de Java y pregunta al docente.

Qué vas a lograr

1. Traducir un modelo de datos relacional a un modelo de clases.
2. Aplicar encapsulamiento, constructores, validaciones, `equals`, `hashCode` y `toString`.
3. Relacionar clases mediante asociación y composición.
4. Usar colecciones (listas y mapas), enumeraciones, excepciones propias y el paquete `java.time`.
5. Separar el programa en capas (modelo, repositorio, servicio, interfaz de consola).
6. Trabajar en equipo con Git y GitHub: ramas, commits y pull requests.

Cómo está organizada

Parte	Contenido	Quién la lee
Secciones 2 a 5	Reglas, caso de estudio, arquitectura y convenciones	Todo el equipo
Sección 6	Git y GitHub paso a paso	Todo el equipo
Secciones 7 a 11	Un módulo por estudiante, con pistas por clase	Cada estudiante lee el suyo y repasa el de quien depende de él
Secciones 12 a 15	Integración, evaluación, puente a JDBC/JPA, glosario	Todo el equipo

Reparto de módulos

Cada integrante construye **un módulo**, definido por las tablas del modelo de datos que le corresponden:

Módulo	Tablas del modelo	Clases principales	Responsabilidad adicional
1 · Clientes y catálogos	`tblCountries`, `tblDocumentsType`, `tblCustomers`	`Country`, `DocumentType`, `Customer`	—
2 · Ubicaciones	`tblCities`, `tblAirports`	`City`, `Airport`	—
3 · Aerolíneas	`tblAirlines`	`Airline`	Aplicación principal (`Main` y `MainMenu`), por ser la tabla más pequeña
4 · Vuelos	`tblFlights`	`Flight`	Reporte de vuelos por aerolínea y mes
5 · Reservas	`tblReservations`, `tblDetailsReservation`	`Reservation`, `ReservationDetail`, `Seat`	Regla de sillas y manifiesto del vuelo

Las piezas compartidas (excepciones, contrato del repositorio, generador de ids, validaciones, lectura de consola) vienen en un **kit base** que entrega el docente (sección 4).

Cómo leer las pistas

Cada clase de cada módulo trae tres tipos de ayuda:

- **Pista 1 (orientación):** qué debes pensar antes de escribir.
- **Pista 2 (estructura):** qué atributos, métodos y reglas debe tener la clase.
- **Pista 3 (detalle):** advertencias sobre errores frecuentes.

Intenta primero con la Pista 1. Pasa a la siguiente solo cuando llevas un buen rato sin avanzar y ya probaste, imprimiste valores y releíste tus apuntes.

Dónde repasar cada tema: el curso POO con Java

Esta guía se apoya en el Curso abierto de POO con Java (JDK 21 LTS, IntelliJ IDEA, Maven y Git). Las señales que verás en toda la guía son:

- 📘 **Repasa en el curso:** enlace a la lección donde se explica el tema.
- 🔎 **Investiga:** el tema no está en el curso; búscalo en la documentación oficial de Java 21 o pregunta en clase.

Tema que necesitarás	Lección del curso	Dónde lo usas
Variables, tipos de dato y constantes	M0 · Variables, tipos de dato y constantes	Todos los módulos
Lectura de datos con `Scanner`	M0 · Operadores y entrada de datos	Menús (capa `ui`)
Validaciones con condicionales	M0 · Condicionales	Constructores y servicios
Menús repetitivos	M0 · Ciclos	Menús
Métodos pequeños y reutilizables	M0 · Modularidad · G1 · Métodos y modularidad	Todos
Maven, Git, Gitflow y trabajo en equipo	G0 · Maven y Git · G0 · Gitflow · G0 · Trabajo colaborativo	Sección 6
Listas	G1 · List y ArrayList	Repositorios, reservas

Tema que necesitarás	Lección del curso	Dónde lo usas
Operaciones guardar, buscar, listar y eliminar	G1 · CRUD en memoria	Todos los repositorios
Mapas	G1 · Map y HashMap	Repositorios, reporte de vuelos
Listas de registros con mapas	G1 · ArrayList con Map	Reportes y agrupaciones
Clases y objetos	G2 · Clases y objetos	Todas las entidades
Encapsulamiento	G2 · Encapsulamiento	Todas las entidades
Constructores (incluida la sobrecarga)	G2 · Constructores	Todas las entidades
Herencia: `extends` y `super`	G3 · extends y super	Jerarquía de excepciones, repositorio base
Sobrescritura: `toString`, `equals`, `hashCode`	G3 · Sobrescritura (@Override)	Todas las entidades
Polimorfismo	G4 · Despacho dinámico · G4 · Colecciones polimórficas	Menú principal, excepciones
Clases y métodos abstractos	G5 · Clase abstracta · G5 · Métodos abstractos	Repositorio en memoria base
Interfaces	G6 · interface e implements · G6 · Clase abstracta vs. interfaz	Contrato del repositorio, `Comparable`, menús

Temas que no están en el curso y debes investigar 🔎: excepciones (creación de las tuyas y uso de `try/catch`), clases de fechas y horas (`LocalDate`, `LocalTime`, `LocalDateTime`, `Duration`), `UUID`, `Comparator` y `Comparable`, enumeraciones, conjuntos (`Set`), genéricos y el concepto de composición.

2. Reglas del ejercicio y uso responsable de la IA

El objetivo es que **tú** consolides los conceptos vistos en clase; por eso la IA se usa poco y solo como apoyo para entender, nunca para producir tu código.

Lo que sí puedes hacer con IA

- Pedir que te **explique un concepto** con un ejemplo distinto al de tu proyecto (por ejemplo, qué es una interfaz).
- Pedir que te **explique un mensaje de error** del compilador o una excepción en tiempo de ejecución.
- Pedir que te haga **preguntas** sobre tu diseño para encontrar huecos.
- Pedir que revise tu **ortografía** de comentarios o del archivo README.

Lo que no debes hacer

- Pedir que genere las clases del módulo, ni siquiera "una base".
- Pegar esta guía completa y pedir la solución.
- Copiar código de un compañero o de internet sin entenderlo línea por línea.
- Subir a GitHub código que no puedas explicar en voz alta.

Regla de oro de la defensa

Cuando el docente integre los módulos, podrá pedirte que expliques cualquier clase de tu módulo: por qué un atributo es privado, por qué usaste una lista y no un mapa, qué pasa si se lanza tal excepción. Si no puedes explicarlo, ese módulo no se considera dominado.

Niveles de ayuda cuando te atascas

Sigue este orden y anota en tu bitácora (sección 13) hasta qué nivel llegaste.

1. **Relee** la pista de la clase en la guía y tus apuntes de clase.
2. **Aísla el problema:** comenta código hasta tener el error más pequeño posible; imprime valores con la consola.
3. **Pregunta a un compañero** de otro módulo, explicándole qué esperabas y qué ocurrió.
4. **Consulta la documentación oficial** de Java (la API de `String`, `ArrayList`, `LocalDate`, etc.).
5. **Pregunta al docente** en clase o por el canal acordado.
6. **Último recurso:** IA, solo para explicar el concepto o el error, como se indicó arriba.

Bitácora personal

Lleva un archivo corto con tres columnas: *qué intenté*, *qué pasó*, *qué aprendí*. Es parte de la evaluación y te sirve para estudiar.

3. El caso de estudio: Airlines-poo

Airlines-poo gestiona **aerolíneas, vuelos entre aeropuertos, clientes y reservas con sus pasajeros**. La base de datos tiene 9 tablas; en Java cada tabla se convierte en una clase y cada clave foránea en una relación entre clases.

De tabla a clase

Tabla (SQL)	Clase Java sugerida	Módulo
tblCountries	`Country`	1
tblDocumentsType	`DocumentType`	1
tblCustomers	`Customer`	1
tblCities	`City`	2
tblAirports	`Airport`	2
tblAirlines	`Airline`	3
tblFlights	`Flight`	4
tblReservations	`Reservation`	5
tblDetailsReservation	`ReservationDetail`	5

Cómo se relacionan (las claves foráneas, contadas en palabras)

- Un **cliente** tiene un tipo de documento y pertenece a un país.
- Un **aeropuerto** está en una ciudad.
- Un **vuelo** pertenece a una aerolínea y tiene dos aeropuertos: el de salida y el de llegada.
- Una **reserva** la hace un cliente para un vuelo.
- Una reserva tiene uno o más **detalles**, uno por pasajero, con su silla.

Tipos de datos: de SQL a Java

En SQL	En Java	Comentario
INT, SMALLINT	`int` , `short`	Evita `short` si te complica; justifica tu decisión

En SQL	En Java	Comentario
VARCHAR(n), CHAR(2)	`String`	La longitud máxima se valida en el constructor o el setter
UNIQUEIDENTIFIER	`UUID`	Se genera con la clase `UUID` de `java.util`
DATE	`LocalDate`	Paquete `java.time`
TIME	`LocalTime`	Paquete `java.time`
NULL permitido	Atributo opcional	Decide si aceptas `null` o un valor por defecto; sé coherente
NOT NULL	Atributo obligatorio	Se valida en el constructor

Reglas que ya trae el modelo de datos

Regla	Dónde se origina
El nombre de una aerolínea no se repite	UNIQUE en tblAirlines
El código de un vuelo (máx. 8 caracteres) no se repite	UNIQUE en tblFlights
El correo de un cliente no se repite	UNIQUE en tblCustomers
El teléfono del cliente es obligatorio (máx. 15)	NOT NULL en tblCustomers
Fecha y hora de salida y llegada son obligatorias	NOT NULL en tblFlights
Los ids de aerolíneas empiezan en 100 y avanzan de 10 en 10	IDENTITY(100,10)
Los ids de ciudades y vuelos empiezan en 1 y avanzan de 1 en 1	IDENTITY(1,1)
Una referencia a otra tabla debe existir	Claves foráneas

Reglas de negocio que el modelo no impone (las agregas tú)

- La llegada no puede ser anterior a la salida.
- El aeropuerto de salida y el de llegada de un vuelo deben ser distintos.

- Una silla (fila + número) no puede asignarse a dos pasajeros en el mismo vuelo.
- Un pasajero no puede aparecer dos veces en la misma reserva.
- Una reserva debe tener al menos un pasajero.

Los procedimientos almacenados, vistos como métodos

El script original traía tres procedimientos. En Java serán métodos de la capa de servicio:

Procedimiento	Se convierte en	Módulo
uspCrearAerolineas	Método que registra una aerolínea	3
uspCrearClientes	Método que registra un cliente	1
uspVuelosPorAerolinea (año, mes)	Método que lista vuelos de un mes, agrupados por aerolínea	4

Los métodos reflejan el CRUD de la base de datos

Todo lo que tu módulo haga con los datos corresponde a una de estas operaciones, que más adelante se convertirán en sentencias SQL:

Operación	Sentencia SQL	Ejemplo en palabras
Create	`INSERT`	Registrar un cliente
Read	`SELECT`	Consultar un cliente por documento, listar todos
Update	`UPDATE`	Cambiar el teléfono de un cliente
Delete	`DELETE`	Eliminar un cliente
Complementarias	`SELECT … WHERE`, `COUNT`, `EXISTS`, `JOIN`, `ORDER BY`	Buscar por apellido, contar, preguntar si existe, listar con datos de otras tablas

Por eso cada módulo de la guía trae un **Mapa CRUD**: una tabla que dice qué método del repositorio y del servicio cubre cada operación y qué sentencia SQL tendrá detrás. Los nombres de los métodos son los mismos en todos los módulos (sección 4).

4. Arquitectura del proyecto y estructura de paquetes

El proyecto se organiza en **cuatro capas**, de modo que cuando llegue JDBC o JPA solo cambie una de ellas.

Capa	Paquete	Responsabilidad	Pregunta que responde
Modelo	`model`	Clases que representan los datos y sus reglas internas	¿Qué es un cliente, un vuelo?
Repositorio	`repository`	Guardar, buscar, listar y eliminar objetos (por ahora en memoria)	¿Dónde viven los datos?
Servicio	`service`	Reglas de negocio que involucran varios objetos	¿Qué se permite hacer?
Interfaz	`ui`	Menús y lectura/escritura en consola	¿Cómo interactúa el usuario?

La regla de dependencia es de arriba hacia abajo: la interfaz usa servicios, los servicios usan repositorios, los repositorios usan el modelo. **Nunca al revés** y nunca saltándose una capa (la interfaz no habla directo con el repositorio).

Estructura de carpetas acordada

El docente entrega el repositorio con esta estructura vacía. No cambies nombres de paquetes sin acordarlo con todo el equipo.

```
airlines-poo/
├── pom.xml
├── README.md
├── .gitignore
└── src/main/java/com/airlinespoo/
    ├── core/          (kit base del docente: excepciones, repositorio base,
generador de ids, utilidades)
    ├── customers/     (Módulo 1: países, tipos de documento y clientes)
    ├── locations/     (Módulo 2: ciudades y aeropuertos)
    ├── airlines/      (Módulo 3: aerolíneas)
    ├── flights/       (Módulo 4: vuelos)
    ├── reservations/  (Módulo 5: reservas y pasajeros)
    └── app/           (Módulo 3: clase principal y menú general)
```

Cada módulo tiene dentro sus propios subpaquetes `model`, `repository`, `service` y `ui`. Así, cada estudiante trabaja en **carpetas distintas** y se reducen los conflictos en Git.

El repositorio en memoria 🔎

Un repositorio es un objeto que esconde *cómo* se guardan los datos. Por ahora guardará en una colección de Java (lista o mapa); en las próximas clases guardará en una base de datos sin que el servicio ni la interfaz se enteren. Para lograrlo se define primero un **contrato** (una interfaz) con las operaciones comunes, y luego una clase que lo implementa en memoria. El docente entrega ese contrato en el kit base; cada módulo lo extiende para sus entidades.

📘 **Repasa en el curso:** G1 · CRUD en memoria (las operaciones del repositorio), G6 · interface e implements (el contrato) y G5 · Clase abstracta (la base compartida). Las lecciones G7 · JDBC y G7 · JPA y Hibernate llegarán después y reemplazarán justamente esta capa.

Preparar el proyecto (lo hace el docente, tú solo lo verificas)

1. Proyecto Maven con JDK 21 LTS (la misma versión del curso).
2. Paquete base `com.airlinespoo`.
3. Verifica que tu IDE importe el proyecto y compile sin errores antes de escribir nada.
4. Verifica que puedes ejecutar la clase principal (aunque solo imprima un saludo).

El kit base del docente (paquete `core`)

Las piezas que usan todos los módulos las entrega el docente en el paquete `core`, ya integradas en `develop`. **Tu trabajo es leerlas, entenderlas y usarlas; no las modifiques.** Si crees que falta algo o hay un error, abre un *Issue* en GitHub.

Esta lista es también la especificación de lo que contiene el kit.

1. Excepciones

Una excepción **base** del sistema y cuatro especializadas que heredan de ella:

Excepción	Cuándo se lanza	Quién la lanza
De validación	Un dato no cumple una regla (vacío, muy largo, fuera de rango)	Modelo y utilidades de validación

Excepción	Cuándo se lanza	Quién la lanza
De duplicado	Ya existe un objeto con ese id o con ese valor único	Servicio
De no encontrado	Se pidió un objeto que no existe	Servicio
De regla de negocio	Una regla entre varios objetos no se cumple (silla ocupada, vuelo ya salido)	Servicio

Todas son *unchecked* 🔎 (no obligan a declarar ni capturar) y llevan mensajes en español. La interfaz de consola las captura; el modelo y los servicios las lanzan.

2. Interfaz de identidad (`Identifiable`)

Contrato mínimo: "todo objeto con identidad ofrece su id". Lo implementan `Country`, `DocumentType`, `Customer`, `City`, `Airport`, `Airline`, `Flight` y `Reservation`. El tipo del id cambia según la entidad (entero, texto, `UUID`), por eso lleva un parámetro de tipo 🔎.

3. Contrato del repositorio (`Repository`)

Sus métodos reflejan el **CRUD de la base de datos** y se llaman igual en todos los módulos:

Método	Equivale a	Comportamiento
`create`	`INSERT`	Guarda un objeto nuevo; falla si el id ya existe
`findById`	`SELECT … WHERE id = ?`	Devuelve el objeto o indica ausencia (con `Optional` 🔎)
`findAll`	`SELECT …`	Devuelve una **copia** de todos los objetos
`update`	`UPDATE`	Reemplaza un objeto existente; falla si no existe
`deleteById`	`DELETE … WHERE id = ?`	Elimina y responde si eliminó algo
`existsById`	`SELECT 1 … WHERE id = ?`	Responde sí o no
`count`	`SELECT COUNT(*)`	Devuelve la cantidad

El repositorio **no** lanza "no encontrado": eso lo decide el servicio, que conoce el contexto. Las búsquedas complementarias de cada módulo siguen este patrón de nombres: `findBy<Criterio>`, `existsBy<Criterio>` y `countBy<Criterio>`.

4. Repositorio base en memoria (`InMemoryRepository`)

Clase **abstracta** que implementa el contrato con un mapa interno. Cada módulo la extiende y le agrega sus búsquedas propias usando un método protegido que devuelve una copia de los valores. Nadie la instancia directamente.

5. Generador de ids (`IdGenerator`)

Se crea indicando el valor de **arranque** y el **paso** (mayor que cero). Entrega el valor actual y avanza. Ofrece una forma de **sincronizarse** con ids ya cargados. Configuración del proyecto: aerolíneas (100, de 10 en 10); ciudades, vuelos, reservas y detalles (1, de 1 en 1).

6. Utilidades de validación (`Validator`)

Clase con **solo métodos estáticos** (no se instancia): texto obligatorio, longitud máxima, valor positivo, rango, objeto no nulo y normalización de texto (quita espacios sobrantes). Todas lanzan la excepción de validación e incluyen el **nombre del campo** en el mensaje.

7. Interfaz `Menu` y lector `ConsoleInput`

`Menu` es el contrato de las pantallas de consola: ofrece un título y se ejecuta hasta que el usuario decide salir. `ConsoleInput` envuelve el teclado (una sola instancia compartida) y ofrece lectura de texto, texto obligatorio, entero en un rango, fecha, hora y confirmación sí/no, **repreguntando** cuando el dato no es válido.

Nombres de los métodos de servicio

Operación	Patrón	Ejemplos
Crear	`create<Entidad>`	`createCustomer`, `createFlight`
Leer uno	`get<Entidad>`	`getAirline`, `getAirport`
Leer todos	`list<Entidades>`	`listCities`, `listFlights`
Actualizar	`update<Entidad>` (o un verbo más específico, como `renameAirline`)	`updateCustomer`

Operación	Patrón	Ejemplos
Eliminar	`delete<Entidad>`	`deleteCity`
Complementarios	Verbo + criterio	`searchCustomersByLastName`, `listFlightsByAirline`, `countFlightsByAirline`, `airlineHasFlights`

Lectura guiada del kit (antes de programar)

Lee cada pieza y responde en tu bitácora, con tus palabras y sin ayuda de IA:

1. ¿Por qué todas las excepciones del sistema heredan de una misma base?
2. ¿Por qué `InMemoryRepository` es abstracta y `Repository` es una interfaz? ¿Qué problema resuelve cada una?
3. ¿Por qué `findAll` devuelve una copia y no la colección interna?
4. ¿Por qué `create` y `update` son operaciones distintas?
5. ¿Qué ventaja tiene un generador configurable (arranque y paso) frente a un contador fijo?
6. ¿Qué ocurre si un modelo valida sus datos con sus propias reglas en lugar de usar `Validator`?

📘 Para entender el diseño del kit repasa G5 · Clase abstracta, G6 · interface e implements, G6 · Clase abstracta vs. interfaz y G3 · extends y super.

Nota para el docente: si prefieres que los estudiantes construyan el kit en lugar de recibirlo, la pieza natural para asignarla es el Módulo 3 (la tabla más pequeña), con una entrega inicial al comienzo de la primera sesión para no bloquear a los demás.

5. Convenciones comunes del equipo

Para que cinco módulos escritos por cinco personas se integren sin dolor, todos siguen las mismas reglas.

Nombres

Elemento	Regla	Ejemplo en palabras
Clases	Inglés, singular, `PascalCase`	Customer, ReservationDetail
Atributos y métodos	Inglés, `camelCase`, verbos en los métodos	firstName, findById
Paquetes	Minúsculas, sin guiones	com.airlinespoo.flights.model
Constantes	MAYÚSCULAS con guion bajo	MAX_FLIGHT_NAME_LENGTH
Comentarios y mensajes al usuario	Español claro	"La llegada no puede ser anterior a la salida"

Los errores de nombre del esquema SQL

En la base de datos hay nombres con errores: `FirtName`, `PassangerFullname`, `PassagerDocument`, `SeatNumer`. **En Java los escribimos bien** (`firstName`, `passengerFullName`, `passengerDocument`, `seatNumber`). Cuando lleguemos a JDBC y JPA aprenderemos a *mapear* el nombre de la columna al nombre del atributo. Anota esta diferencia en tu bitácora: es una excelente pregunta de examen.

Reglas de diseño de clases

1. **Atributos siempre privados.** Se accede mediante métodos públicos.
2. **El constructor deja el objeto en un estado válido.** Si falta un dato obligatorio o supera la longitud máxima, el constructor rechaza la creación lanzando una excepción.
3. **Los identificadores no se modifican** después de creados: no tienen setter.
4. **Solo hay setters donde el negocio permite cambiar el dato**, y cada setter valida igual que el constructor.
5. **`toString` legible** para depurar e imprimir listados; no debe incluir datos sensibles.
6. **`equals` y `hashCode` basados en la identidad del negocio** (normalmente el id), no en todos los atributos. 🔎
7. **Las relaciones se modelan con referencias a objetos**, no con ids sueltos: un `Flight` guarda su `Airline`, no un número de aerolínea. Esto es POO; el id numérico reaparecerá cuando persistamos.

8. **Las colecciones nunca se exponen directamente**: si una clase devuelve su lista interna, quien la reciba podrá dañarla. Devuelve una vista de solo lectura o una copia. 🔎
9. **Ninguna clase de `model`, `repository` o `service` imprime en consola ni lee del teclado.** Solo la capa `ui` interactúa con el usuario.

📘 **Repasa en el curso:** G2 · Encapsulamiento, G2 · Constructores y G3 · Sobrescritura (@Override), donde `toString`, `equals` y `hashCode` son métodos que se sobrescriben.

Validaciones y excepciones 🔎

- Las reglas se validan en el **modelo** (reglas de un solo objeto) o en el **servicio** (reglas entre varios objetos).
- Los errores se comunican con **excepciones propias** incluidas en el kit base del docente (por ejemplo, de validación, de duplicado, de no encontrado). No uses códigos de retorno ni imprimas el error y sigas.
- La capa `ui` captura las excepciones y muestra un mensaje amable.
- No captures excepciones solo para ocultarlas: o las manejas con sentido o las dejas subir.

Calidad del código

- Un método hace **una sola cosa** y cabe en una pantalla.
- Sin números ni textos "mágicos" repetidos: usa constantes.
- Comentarios de documentación (Javadoc) en cada clase pública y en los métodos públicos.
- Sin código comentado ni archivos de prueba sueltos en el repositorio.
- Una clase pública por archivo, con el mismo nombre de la clase.
- Antes de cada commit: el proyecto compila y el programa se ejecuta.

Pruebas manuales

Por ahora probarás con una clase `ui` y la clase principal. Cada módulo trae una **tabla de casos de prueba** (datos de entrada y resultado esperado) que debes ejecutar a mano y marcar como superada. En clases posteriores veremos pruebas automáticas.

6. Git y GitHub: trabajar cinco personas sobre un mismo proyecto

Git guarda el historial de tu proyecto en tu computador y **GitHub** aloja la copia compartida del equipo. Aquí aplicas lo que ya viste en el curso: el modelo **Gitflow** con ramas `feature/*` que se integran en `develop`, y el trabajo colaborativo con Pull Requests.

📘 **Repasa antes de empezar (curso POO con Java):**

- G0 · Herramientas de desarrollo
- G0 · Maven y Git
- G0 · Gitflow: gestión de ramas
- G0 · Trabajo colaborativo: caso con 3 desarrolladores (incluye un conflicto real y su solución)

Vocabulario mínimo

Término	Qué significa	Analogía
Repositorio (repo)	Carpeta del proyecto con todo su historial	El cuaderno completo
Commit	Una "foto" guardada del estado del proyecto, con un mensaje	Una página fechada y firmada
Rama (branch)	Línea de trabajo paralela e independiente	Tu propia copia para experimentar
Remoto (`origin`)	La copia del repo en GitHub	La carpeta compartida
Clonar	Descargar el repo de GitHub por primera vez	Fotocopiar el cuaderno
Push	Subir tus commits a GitHub	Entregar tus páginas
Pull	Traer a tu computador lo nuevo de GitHub	Recoger lo que otros entregaron
Pull request (PR)	Solicitud para que tu rama se integre en otra, con revisión	"Profe, revise mi trabajo, por favor"
Merge	Unir los cambios de una rama en otra	Pegar tus páginas en el cuaderno oficial

Término	Qué significa	Analogía
Conflicto	Dos personas cambiaron el mismo lugar del código	Dos personas corrigieron la misma frase

Las ramas de este proyecto (Gitflow)

Rama	Nace de	Se integra en	Quién la usa	Ejemplo
`main`	—	—	Solo el docente. Refleja versiones publicadas y etiquetadas	`main`
`develop`	`main`	—	Aquí se integran los módulos terminados	`develop`
`feature/*`	`develop`	`develop`	**Tú**: una rama por módulo	`feature/modulo-1-clientes`
`release/*`	`develop`	`main` y `develop`	Solo el docente, al publicar una versión	`release/1.0.0`
`hotfix/*`	`main`	`main` y `develop`	Solo si hay un error crítico ya publicado	`hotfix/1.0.1`

Tú solo creas ramas `feature/*`. Todo tu trabajo parte de `develop` y vuelve a `develop` mediante Pull Request.

Parte A. Preparación (se hace una sola vez)

1. **Crea tu cuenta de GitHub** y envía tu usuario al docente para que te agregue como colaborador.
2. **Acepta la invitación** que llega a tu correo o a tus notificaciones de GitHub.
3. **Comprueba Git** con `git --version` (la instalación está en G0 · Herramientas).
4. **Configura tu identidad** (debe coincidir con tu cuenta, así tus commits quedan a tu nombre):

```
git config --global user.name "Tu Nombre Apellido"
git config --global user.email "tu_correo@ejemplo.com"
```

5. **Configura la autenticación.** GitHub ya no acepta contraseña desde la terminal. Elige una opción y pide ayuda al docente si te trabas: un *Personal Access Token*, una llave SSH, o el inicio de sesión integrado de IntelliJ IDEA o de GitHub Desktop.
6. **Clona el repositorio** (la URL te la da el docente) y cámbiate a `develop`:

```
git clone <url-del-repositorio>
cd airlines-poo
git checkout develop
```

7. Abre la carpeta en IntelliJ IDEA, deja que Maven descargue lo necesario, compila y ejecuta la clase principal para confirmar que todo funciona.

Parte B. Reglas del equipo

- **Nadie hace push directo a `main` ni a `develop`.** El docente protege ambas ramas; el trabajo llega a `develop` solo mediante Pull Request.
- **Una rama `feature/modulo-N-tema` por módulo**: por ejemplo `feature/modulo-1-clientes` o `feature/modulo-4-vuelos`.
- **Solo trabajas dentro de las carpetas de tu módulo.** Si necesitas cambiar algo de otro módulo o del núcleo, lo pides con un *Issue* de GitHub; no lo modificas tú.
- **Commits pequeños y frecuentes**: uno por cada avance con sentido (una clase terminada, una validación nueva).
- **Nunca uses `git push --force`.** Reescribe el historial y puede borrar el trabajo de otros.
- **No subas archivos generados**: carpeta `target/`, archivos `.class`, configuración del IDE, archivos del sistema operativo.

Parte C. Ciclo de trabajo de cada módulo

Repite este ciclo hasta terminar tu módulo.

1. **Actualiza tu copia de `develop`** antes de empezar:

```
git checkout develop
git pull origin develop
```

2. **Crea tu rama feature** y cámbiate a ella:

```
git checkout -b feature/modulo-1-clientes
```

3. **Programa** una pieza pequeña (por ejemplo, la clase `Country`) y verifica que compila.
4. **Revisa qué cambió:**

```
git status
git diff
```

5. **Prepara los archivos** que quieres guardar y haz el commit. En el curso usaste `git add .`; aquí es mejor agregar archivo por archivo (o carpeta por carpeta) para no subir algo de otro módulo por accidente:

```
git add src/main/java/com/airlinespoo/customers/model/Country.java
git commit -m "feature: agrega clase Country con validaciones"
```

6. **Sube tu rama** a GitHub (la primera vez Git te sugiere el comando completo con `--set-upstream`):

```
git push origin feature/modulo-1-clientes
```

7. Repite los pasos 3 a 6 con cada avance.
8. Cuando el módulo esté completo, abre el **Pull Request** (Parte E).

Parte D. Cómo escribir buenos commits

Un mensaje de commit responde a la pregunta "¿qué hace este cambio?". Seguimos el formato del curso: `tipo: descripción corta en español`.

Tipo	Cuándo se usa	Ejemplo
`feature`	Agregas algo nuevo	`feature: agrega clase Flight`
`fix`	Corriges un error	`fix: corrige validación del correo del cliente`
`refactor`	Mejoras el código sin cambiar su comportamiento	`refactor: extrae validación de nombre en ciudades`
`docs`	Cambias comentarios o documentación	`docs: documenta la clase ReservationDetail`
`test`	Agregas casos o clases de prueba	`test: agrega casos del generador de ids`

Tipo	Cuándo se usa	Ejemplo
`merge`	Resuelves un conflicto	`merge: resuelve conflicto con develop en el menú`

Malos mensajes: "cambios", "listo", "arreglé cosas", "asdf". Buenos mensajes: los que un compañero entiende sin abrir el archivo.

Parte E. Pull Request: entregar tu módulo

En el curso viste que en un equipo real la integración se hace con Pull Requests (G0 · Trabajo colaborativo, apartado final). Estos son los pasos:

1. Haz el último `push` de tu rama.
2. En GitHub aparecerá un botón **"Compare & pull request"**. Púlsalo. Verifica que dice: *base:* `develop` ← *compare: tu rama* `feature/...`. **Si dice `main`, cámbialo.**
3. **Título:** `Módulo 1: Clientes y catálogos`.
4. **Descripción:** qué clases entregas, qué reglas validaste, qué casos de prueba superaste y qué dudas tienes.
5. Marca como revisor al **compañero asignado** y al docente.
6. Espera la revisión. Si te piden cambios, los haces **en la misma rama** con nuevos commits y haces push: el PR se actualiza solo.
7. El docente integra tu PR en `develop` con un commit de fusión (equivale a `git merge --no-ff` del curso, que deja constancia en el historial de que existió una rama de funcionalidad). Después **no borres tu rama** hasta que él lo confirme.

Parte F. Revisión entre pares

Antes de que el docente revise, un compañero lee tu PR. Quien revisa mira:

- ¿El código compila y se entiende?
- ¿Respeta las convenciones de la sección 5?
- ¿Los nombres dicen lo que hacen?
- ¿Hay validaciones y mensajes claros?
- ¿Encontró algún caso de prueba que falle?

Los comentarios se escriben con respeto y señalando el *qué* y el *porqué*. Revisar el código de otro también es una forma de estudiar.

Parte G. Mantener tu rama al día

Mientras trabajas, `develop` puede recibir módulos de otros compañeros. Para traerlos a tu rama:

```
git checkout develop
git pull origin develop
git checkout feature/modulo-4-vuelos
git merge develop
```

Hazlo al empezar cada sesión de trabajo y antes de abrir tu PR.

Parte H. Resolver un conflicto

Un conflicto no es un error de Git: es Git pidiendo ayuda humana porque dos personas cambiaron el mismo lugar del código. Si quieres ver uno resuelto paso a paso, repasa el Paso 4 de G0 · Trabajo colaborativo. Aparece al hacer `merge` o al abrir un PR. Sigue estos pasos con calma:

1. Ejecuta `git status`: te muestra los archivos en conflicto.
2. Abre cada archivo. Verás marcas con `<<<<<<<`, `=======` y `>>>>>>>` que encierran las dos versiones.
3. **Decide qué queda**: tu versión, la otra o una mezcla. Borra las marcas.
4. Compila y ejecuta el programa.
5. Ejecuta `git add` sobre cada archivo resuelto y `git commit` con un mensaje de tipo `merge`.
6. Haz `git push`.

Si el conflicto involucra archivos de otro módulo, **no los resuelvas tú solo**: avisa al docente, porque significa que alguien salió de su carpeta.

Errores frecuentes y cómo salir

Situación	Causa probable	Qué hacer
`Permission denied` o `403` al hacer push	No aceptaste la invitación o falla la autenticación	Revisa la invitación y tu token o llave
`rejected ... non-fast-forward`	Tu rama local está desactualizada	`git pull origin <tu-rama>` y luego push

Situación	Causa probable	Qué hacer
Creaste tu rama desde `main` en vez de `develop`	Olvidaste el paso de actualizar `develop`	Avisa al docente antes de abrir el PR
Hiciste commits en `develop` por error	Olvidaste cambiar de rama	Avisa al docente **antes** de hacer push; no improvises
Subiste `target/` o `.idea/`	Falta o falla `.gitignore`	Pide ayuda para sacarlos del seguimiento y agrégalos al `.gitignore`
Hiciste commit con el nombre equivocado	Identidad sin configurar	Configura `user.name` y `user.email` y continúa
Quieres descartar cambios sin guardar	Experimento fallido	`git restore <archivo>` (borra tus cambios de ese archivo; úsalo con cuidado)
Estás perdido y no sabes en qué rama estás	Normal al inicio	`git status` y `git branch` te lo dicen

Hoja de comandos

Para…	Comando
Ver el estado	`git status`
Ver cambios sin guardar	`git diff`
Ver historial resumido	`git log --oneline`
Ver ramas	`git branch`
Crear y entrar a una rama	`git checkout -b <nombre>`
Cambiar de rama	`git checkout <nombre>`
Preparar un archivo	`git add <archivo>`
Guardar un commit	`git commit -m "tipo: mensaje"`
Subir tu rama	`git push origin <rama>`
Traer lo nuevo	`git pull origin <rama>`

Para…	Comando
Unir `develop` a tu rama	`git merge develop`

Archivo `.gitignore`

El docente lo incluirá en el repositorio. Debe ignorar, como mínimo: la carpeta de compilación (`target/`), los archivos `.class` , las carpetas del IDE (`.idea/` , `.vscode/` , `.settings/`), los archivos `.iml` y los archivos del sistema (`.DS_Store` , `Thumbs.db`). Comprueba con `git status` que no aparezcan.

Checklist antes de cada push

- [ ] El proyecto compila.
- [ ] La clase principal se ejecuta.
- [ ] Estoy en **mi rama** `feature/...` (`git branch`), no en `main` ni en `develop` .
- [ ] `git status` solo muestra archivos de **mi módulo**.
- [ ] El mensaje del commit explica el cambio y usa el formato `tipo: descripción` .
- [ ] Traje lo último de `develop` hoy.

Para el docente: configuración del repositorio en GitHub

1. Crear el repositorio con README, `.gitignore` para Java/Maven y la estructura de paquetes vacía en `main` ; crear `develop` desde `main` .
2. Marcar `develop` como **rama por defecto** (Settings → Branches), de modo que los PR de los estudiantes apunten a ella automáticamente.
3. Agregar a los cinco estudiantes como **colaboradores**.
4. Proteger `main` y `develop` (Settings → Branches): exigir pull request y al menos una aprobación; impedir push directo y *force push*.
5. Crear **un Issue por módulo** con su lista de tareas y asignarlo al estudiante. Opcional: un tablero de Projects con columnas *Por hacer*, *En curso*, *En revisión*, *Integrado*.
6. Crear una plantilla de PR con las preguntas de la Parte E.
7. Al cerrar el ejercicio: abrir `release/1.0.0` desde `develop` , ajustar la versión en `pom.xml` , integrarla en `main` con `--no-ff` , etiquetar `v1.0.0` y volver a integrarla en `develop` , tal como se muestra en el Paso 7 de G0 · Trabajo colaborativo.

7. Módulo 1: Clientes y catálogos de identificación

Estudiante A · Rama feature/`modulo-1-clientes` · Paquete `com.airlinespoo.customers`

Construyes la gestión de clientes y los dos catálogos que los describen: país y tipo de documento. Es el módulo con las clases más sencillas para empezar y el que más practica encapsulamiento y validaciones.

Qué practicas

Clases y objetos · atributos privados · constructores · getters y setters con validación · `equals`, `hashCode` y `toString` · asociación entre clases · colecciones (`List`, `Map`) · ordenamiento con `Comparator` 🔎 · excepciones propias.

📘 **Repasa en el curso:**

- G2 · Clases y objetos: cómo pasar de tabla a clase.
- G2 · Encapsulamiento: atributos privados y setters que validan.
- G2 · Constructores: constructores que dejan el objeto válido.
- G3 · Sobrescritura (@Override): `toString`, `equals` y `hashCode`.
- G1 · List y ArrayList, G1 · Map y HashMap y G1 · CRUD en memoria: repositorios.
- M0 · Operadores y entrada de datos y M0 · Ciclos: el menú de consola.

🔎 **Investiga (no está en el curso):** excepciones propias, `Comparator` para ordenar y expresiones regulares básicas para validar correo y teléfono.

Dependencias

- **Necesitas del kit base del docente:** las excepciones comunes y el contrato del repositorio. Hasta que lleguen a `develop`, avanza con `Country`, `DocumentType` y `Customer` (el modelo no depende de nadie).
- **Quién te necesita:** el Módulo 5 usa tu `Customer` para las reservas.

Orden recomendado de construcción

1. `Country` → 2. `DocumentType` → 3. `Customer` → 4. Repositorios → 5. Servicio → 6. Interfaz de consola → 7. Datos de ejemplo.

Clase `Country`

Atributo	Tipo sugerido	Regla
id	numérico pequeño	Obligatorio, positivo, no cambia
name	texto	Obligatorio, no vacío, máximo 20 caracteres

- **Pista 1:** piensa en qué datos *necesita* un país para existir. ¿Tiene sentido un país sin nombre?
- **Pista 2:** constructor con los dos datos que valida ambos; solo getters para el id; getter y setter para el nombre (el setter valida igual que el constructor); `toString` que muestre id y nombre; `equals` / `hashCode` por id.
- **Pista 3:** error común: validar en el constructor pero olvidar el setter. Otro: comparar textos con el operador de igualdad en lugar de `equals`.

Clase `DocumentType`

Atributo	Tipo sugerido	Regla
id	texto de 2 caracteres	Obligatorio, exactamente 2 caracteres, mayúsculas (por ejemplo CC, TI, PA)
description	texto	Obligatorio, máximo 20 caracteres

- **Pista 1:** el id aquí no es un número sino un código corto. ¿Qué pasa si alguien escribe "cc" en minúsculas? Decide cómo normalizarlo.
- **Pista 2:** misma estructura que `Country`. Normaliza el código (por ejemplo, convirtiéndolo a mayúsculas y quitando espacios) dentro del constructor.
- **Pista 3:** 🔎 **Pregunta de diseño:** ¿esto debería ser una clase o una enumeración (`enum`)? Anota ventajas y desventajas de cada opción en tu bitácora. Para esta entrega usa clase, porque proviene de una tabla; la enumeración es un reto opcional.

Clase `Customer`

Atributo	Tipo sugerido	Regla
id	texto	Obligatorio, máximo 20 caracteres, no cambia (es el documento de identidad)

Atributo	Tipo sugerido	Regla
firstName	texto	Obligatorio, máximo 30
lastName	texto	Obligatorio, máximo 30
documentType	objeto `DocumentType`	Obligatorio
telephoneNumber	texto	Obligatorio, máximo 15, solo dígitos (opcionalmente con `+` al inicio)
email	texto	Opcional; si existe, máximo 80 y con formato básico válido
country	objeto `Country`	Obligatorio
address	texto	Opcional, máximo 40

- **Pista 1:** fíjate en que el cliente *tiene* un tipo de documento y *pertenece* a un país: son relaciones, y se guardan como **objetos**, no como códigos sueltos.
- **Pista 2:** constructor con los datos obligatorios; los opcionales (correo, dirección) se asignan con setters. Agrega un método que devuelva el nombre completo. `equals` / `hashCode` por id. `toString` sin mostrar datos sensibles innecesarios.
- **Pista 3:** errores comunes: aceptar texto con solo espacios como "nombre válido"; olvidar que el correo opcional puede ser nulo antes de validarlo; duplicar la validación de longitud en varios sitios (extráela a un método privado). 🔎 Piensa cómo manejarías un atributo opcional sin llenar tu código de comprobaciones de nulo.

Repositorios

Crea un repositorio en memoria por cada entidad: países, tipos de documento y clientes. Cada uno implementa el contrato del kit base.

- **Pista 1:** ¿qué colección te permite buscar un cliente por documento sin recorrer todos? Compara una lista con un mapa y justifica.
- **Pista 2:** operaciones mínimas: guardar, buscar por id, listar todos, eliminar, saber si existe. El de clientes agrega buscar por correo.
- **Pista 3:** 🔎 el método que lista no debe devolver la colección interna; devuelve una copia o una vista de solo lectura. Piensa qué ocurriría si el servicio la modificara.

Servicio `CustomerService`

Representa lo que hacía el procedimiento `uspCrearClientes`, con reglas añadidas.

Operación	Reglas que debe cumplir
Registrar cliente	El país y el tipo de documento deben existir; el id no puede repetirse; el correo, si viene, no puede repetirse
Consultar por documento	Si no existe, lanza la excepción de "no encontrado"
Actualizar contacto	Solo teléfono, correo y dirección; el correo nuevo no puede pertenecer a otro cliente
Listar clientes	Ordenados por apellido y luego por nombre 🔎
Buscar por apellido	Coincidencia parcial sin distinguir mayúsculas
Eliminar	Por ahora se permite; en la integración se limitará si el cliente tiene reservas

- **Pista 1:** el servicio *coordina*: pide al repositorio, comprueba reglas, lanza excepciones. No imprime nada.
- **Pista 2:** el servicio recibe los repositorios por su **constructor** (no los crea él mismo). Esto se llama inyección de dependencias manual y será clave para JDBC/JPA. 🔎
- **Pista 3:** error común: validar la unicidad del correo comparando con el propio cliente que se está actualizando (se "choca" consigo mismo).

Mapa CRUD del módulo

Cada método de tus servicios equivale a una operación que luego será SQL. Los repositorios ofrecen las **operaciones comunes** del kit base (`create`, `findById`, `findAll`, `update`, `deleteById`, `existsById`, `count`, ver sección 4) más las búsquedas complementarias que indica la tabla. Usa **estos nombres**, iguales a los de los demás módulos.

Entidad	Operación	En la base de datos	Servicio	Reglas
`Country`	Crear	`INSERT` en `tblCountries`	`createCountry`	Id positivo y no repetido; nombre válido

Entidad	Operación	En la base de datos	Servicio	Reglas
`Country`	**R**eer uno / todos	`SELECT` por id / `SELECT … ORDER BY Name`	`getCountry` / `listCountries`	No encontrado → excepción; lista ordenada por nombre
`Country`	**U**pdate	`UPDATE … SET Name`	`updateCountry`	Nombre válido
`Country`	**D**elete	`DELETE`	`deleteCountry`	Rechazo si algún cliente pertenece a ese país
`DocumentType`	**C**rear	`INSERT` en `tblDocumentsType`	`createDocumentType`	Código de 2 caracteres normalizado y no repetido
`DocumentType`	**R**eer uno / todos	`SELECT` por id / `SELECT`	`getDocumentType` / `listDocumentTypes`	No encontrado → excepción
`DocumentType`	**U**pdate	`UPDATE … SET Description`	`updateDocumentType`	Descripción válida (máx. 20)
`DocumentType`	**D**elete	`DELETE`	`deleteDocumentType`	Rechazo si algún cliente usa ese tipo
`Customer`	**C**rear	`INSERT` en `tblCustomers` (hacía `uspCrearClientes`)	`createCustomer`	País y tipo de documento existen; documento y correo no repetidos
`Customer`	**R**eer uno	`SELECT … WHERE customerId = ?`	`getCustomer`	No encontrado → excepción
`Customer`	**R**eer todos	`SELECT … ORDER BY Lastname, FirtName`	`listCustomers`	Ordenados por apellido y luego por nombre
`Customer`	**U**pdate	`UPDATE tblCustomers SET …`	`updateCustomer`	Todo menos el documento; el correo no puede pertenecer a otro cliente
`Customer`	**D**elete	`DELETE`	`deleteCustomer`	Por ahora se permite; en la fase de integración se impedirá si tiene reservas
`Customer`	Complementario	`SELECT … WHERE Email = ?`	`getCustomerByEmail`	Opcional: devuelve ausencia si no existe
`Customer`	Complementario	`SELECT … WHERE Lastname LIKE ?`	`searchCustomersByLastName`	Coincidencia parcial sin distinguir mayúsculas
`Customer`	Complementario	`SELECT … WHERE CountryId = ?`	`listCustomersByCountry`	País existente
`Customer`	Complementario	`SELECT … WHERE TypeId = ?`	`listCustomersByDocumentType`	Tipo existente
`Customer`	Complementario	`SELECT 1 … WHERE CountryId = ?`	(uso interno)	Sirve para impedir eliminar países y tipos de documento en uso
`Customer`	Complementario	`SELECT COUNT(*)`	`countCustomers`	Total de clientes

- **Pista 1:** `create` del repositorio falla si el id ya existe; `update` falla si no existe. No los mezcles: en SQL son sentencias distintas.
- **Pista 2:** si un método del servicio te parece demasiado largo, divídelo en pasos con nombre propio (validar existencias, validar unicidad, guardar).
- **Pista 3:** anota en tu bitácora, para cada fila, la sentencia SQL equivalente con tus propias palabras.

Interfaz `CustomerMenu`

Menú de consola: registrar, consultar, actualizar contacto, listar, buscar, eliminar, volver. Lee datos del teclado, llama al servicio y muestra resultados.

- **Pista 1:** separa claramente "pedir datos" de "llamar al servicio".
- **Pista 2:** captura las excepciones del servicio y muestra mensajes en español; el menú nunca debe terminar el programa por un dato mal digitado.
- **Pista 3:** error clásico con `Scanner`: leer un número y luego un texto deja un salto de línea pendiente. Investiga el síntoma y cómo lo evitas (por ejemplo, leyendo siempre líneas completas y convirtiendo después).

Datos de ejemplo

Crea una clase pequeña que cargue 5 países y 4 tipos de documento al iniciar, y 3 clientes de muestra. Así los otros módulos pueden probar sin escribir datos cada vez.

Casos de prueba que debes superar

#	Acción	Resultado esperado
1	Crear país con nombre de 21 caracteres	Rechazo con mensaje claro
2	Crear tipo de documento con id "C"	Rechazo (debe tener 2 caracteres)
3	Crear tipo de documento con id "cc"	Se guarda como "CC"
4	Registrar cliente con nombre vacío	Rechazo
5	Registrar cliente con teléfono "abc123"	Rechazo
6	Registrar dos clientes con el mismo documento	El segundo es rechazado
7	Registrar dos clientes con el mismo correo	El segundo es rechazado

#	Acción	Resultado esperado
8	Registrar cliente con un país inexistente	Rechazo
9	Consultar un documento que no existe	Mensaje "no encontrado"
10	Listar con 3 clientes	Salen ordenados por apellido
11	Actualizar correo a uno ya usado por otro	Rechazo
12	Actualizar correo al mismo que ya tenía	Se acepta

Preguntas para tu bitácora

1. ¿Por qué el id de `Customer` no tiene setter?
2. ¿Qué diferencia hay entre `==` y `equals` al comparar dos clientes con el mismo documento?
3. ¿Qué ventaja tiene guardar el `Country` como objeto y no solo su código?
4. ¿Qué pasaría si `equals` se basara en todos los atributos y alguien cambia el teléfono de un cliente guardado en un `HashMap`?

Retos opcionales

- Convierte `DocumentType` en una enumeración y compara el resultado.
- Impide registrar clientes menores de edad (necesitarás agregar fecha de nacimiento: discútelo antes con el docente).
- Agrega un método que devuelva los clientes de un país dado.

Plan de commits sugerido

1. `feature: agrega Country y DocumentType`
2. `feature: agrega Customer con validaciones`
3. `feature: agrega repositorios en memoria`
4. `feature: agrega CustomerService`
5. `feature: agrega menú de clientes y datos de ejemplo`

8. Módulo 2: Ubicaciones (ciudades y aeropuertos)

Estudiante B · Rama feature/`modulo-2-ubicaciones` · Paquete `com.airlinespoo.locations`

Construyes dónde están los aeropuertos. Es un módulo pequeño pero **estratégico**: el Módulo 4 (vuelos) no puede avanzar del todo sin tu clase `Airport`. Por eso tu primera entrega debe ser rápida y estable.

Qué practicas

Identificadores generados por el programa · `UUID` · dos formas de construir un objeto (nuevo y ya existente) · sobrecarga de constructores 🔎 · asociación unidireccional · búsquedas en colecciones · normalización de texto · excepciones propias.

📘 **Repasa en el curso:**

- G2 · Constructores: sobrecarga, para las dos formas de construir un `Airport`.
- G2 · Clases y objetos y G2 · Encapsulamiento.
- G1 · Map y HashMap, G1 · CRUD en memoria y G1 · ArrayList con Map: repositorios y búsquedas.
- G3 · Sobrescritura (@Override): `equals` y `hashCode`.

🔎 **Investiga (no está en el curso):** la clase `UUID`, los métodos de fábrica estáticos y los métodos de `String` para normalizar texto.

Dependencias

- **Necesitas del kit base del docente:** el generador de identificadores numéricos, las excepciones y el contrato del repositorio.
- **Quién te necesita:** el Módulo 4 usa tu `Airport` como salida y llegada de cada vuelo.

Entrega anticipada

En cuanto `City` y `Airport` compilen y estén validadas, haz su primer push y abre un PR **solo con esas dos clases**. Luego sigues con repositorios, servicio y menú en la misma rama o en una nueva rama de continuación (feature/`modulo-2-ubicaciones-parte-2`).

Clase `City`

Atributo	Tipo sugerido	Regla
id	entero	Lo asigna el programa (arranca en 1 y sube de 1 en 1); no cambia
name	texto	Obligatorio, no vacío, máximo 20 caracteres

- **Pista 1:** en la base de datos el motor numera las ciudades por ti. Aquí, ¿quién lo hará? Revisa qué ofrece el kit base.
- **Pista 2:** constructor con id y nombre; getters; setter del nombre con validación; `equals` / `hashCode` por id; `toString` con el nombre.
- **Pista 3:** decide si "Bogotá" y "bogotá " son la misma ciudad. Define una regla de normalización (espacios sobrantes, mayúsculas) y aplícala siempre en el mismo método.

Clase `Airport`

Atributo	Tipo sugerido	Regla
id	`UUID`	Identificador universal; no cambia
name	texto	Obligatorio, máximo 50 caracteres
city	objeto `City`	En la base de datos es opcional; en el negocio lo haremos **obligatorio** (justifícalo en tu bitácora)

- **Pista 1:** un `UUID` no se numera: se genera aleatoriamente con una clase de Java. ¿Dónde conviene generarlo: dentro del aeropuerto o en el servicio?
- **Pista 2:** 🔎 ofrece **dos formas de construir** el objeto: una para un aeropuerto *nuevo* (genera un id) y otra para uno *existente* que ya tiene id (por ejemplo, el que más adelante traeremos de la base de datos). Esto es sobrecarga de constructores o métodos de fábrica estáticos.
- **Pista 3:** error común: generar el `UUID` dentro de `equals` o `toString` por descuido. Otro: permitir cambiar la ciudad a `null` con el setter.

Decisión de diseño: ¿la ciudad conoce sus aeropuertos?

En el modelo de datos solo el aeropuerto apunta a la ciudad (clave foránea). Tienes dos caminos: (a) la ciudad guarda una lista de sus aeropuertos, o (b) solo el aeropuerto referencia a la ciudad y las búsquedas se hacen en el repositorio. **Implementa (b)** y explica en tu bitácora por qué es más simple de mantener y más cercano a la base de datos. Menciona qué problema aparece con (a) cuando se elimina un aeropuerto.

Repositorios

Uno para ciudades y otro para aeropuertos, ambos implementando el contrato del kit base.

- **Pista 1:** ¿qué búsquedas necesitarás? Por id, por nombre de ciudad, por ciudad, por nombre de aeropuerto.
- **Pista 2:** el de aeropuertos agrega "listar los de una ciudad" y "buscar por nombre".
- **Pista 3:** 🔎 si el aeropuerto se busca por `UUID`, piensa qué tipo de clave usarás en el mapa y por qué `equals` / `hashCode` de `UUID` ya funcionan bien.

Servicio `LocationService`

Operación	Reglas que debe cumplir
Registrar ciudad	Nombre no repetido (sin distinguir mayúsculas); el id se pide al generador
Registrar aeropuerto	La ciudad debe existir; no puede haber dos aeropuertos con el mismo nombre en la misma ciudad
Consultar aeropuerto por id	Si no existe, excepción de "no encontrado"
Listar aeropuertos de una ciudad	Ordenados por nombre; si la ciudad no existe, excepción
Renombrar ciudad o aeropuerto	Respeta las mismas reglas de unicidad
Eliminar ciudad	Solo si no tiene aeropuertos
Eliminar aeropuerto	Por ahora se permite; en la integración se limitará si tiene vuelos

- **Pista 1:** la regla "eliminar ciudad solo si no tiene aeropuertos" refleja el efecto de una clave foránea. Es tu primer contacto con la **integridad referencial** desde Java.
- **Pista 2:** el servicio recibe los repositorios y el generador por el constructor.
- **Pista 3:** error común: comparar nombres sin normalizar, de modo que "Medellín" y "medellín" parezcan distintos.

Mapa CRUD del módulo

Cada método de tus servicios equivale a una operación que luego será SQL. Los repositorios ofrecen las **operaciones comunes** del kit base (`create`, `findById`, `findAll`, `update`, `deleteById`, `existsById`, `count`, ver sección 4) más las búsquedas complementarias que indica la tabla. Usa **estos nombres**.

Entidad	Operación	En la base de datos	Servicio	Reglas
`City`	**C**rear	`INSERT` en `tblCities`	`createCity`	Nombre válido y no repetido (sin distinguir mayúsculas); id del generador
`City`	**R**eer uno	`SELECT … WHERE CityId = ?`	`getCity`	No encontrada → excepción
`City`	**R**eer todas	`SELECT … ORDER BY name`	`listCities`	Ordenadas por nombre
`City`	**U**pdate	`UPDATE … SET name`	`updateCity`	Misma regla de unicidad; puede quedar igual a la propia
`City`	**D**elete	`DELETE`	`deleteCity`	Rechazo si tiene aeropuertos

Entidad	Operación	En la base de datos	Servicio	Reglas
`City`	Complementario	`SELECT … WHERE name LIKE ?`	`searchCitiesByName`	Coincidencia parcial
`Airport`	**C**rear	`INSERT` en `tblAirports`	`createAirport`	La ciudad existe; nombre no repetido en esa ciudad; el `UUID` se genera al crear
`Airport`	**R**eer uno	`SELECT … WHERE AirportId = ?`	`getAirport`	No encontrado → excepción
`Airport`	**R**eer todos	`SELECT … ORDER BY name`	`listAirports`	Ordenados por nombre
`Airport`	**U**pdate	`UPDATE … SET name, CityId`	`updateAirport`	Mismas reglas de unicidad; la ciudad nueva debe existir
`Airport`	**D**elete	`DELETE`	`deleteAirport`	Por ahora se permite; en la fase de integración se impedirá si tiene vuelos
`Airport`	Complementario	`SELECT … WHERE CityId = ?`	`listAirportsByCity`	Ordenados por nombre; si la ciudad no existe, excepción
`Airport`	Complementario	`SELECT … WHERE name LIKE ?`	`searchAirportsByName`	Coincidencia parcial

Entidad	Operación	En la base de datos	Servicio	Reglas
`Airport`	Complementario	`SELECT COUNT(*) … WHERE CityId = ?`	`countAirportsByCity`	Total por ciudad
`Airport`	Complementario	`SELECT 1 … WHERE CityId = ?`	(uso interno)	Sirve para impedir eliminar ciudades con aeropuertos

- **Pista 1:** `create` del repositorio falla si el id ya existe; `update` falla si no existe.
- **Pista 2:** para `Airport`, la operación `update` recibe un objeto *existente* (con su `UUID`), mientras que `create` recibe uno *nuevo*. Eso refuerza la razón de las dos formas de construirlo.
- **Pista 3:** anota en tu bitácora la sentencia SQL equivalente de cada fila.

Interfaz `LocationMenu`

Opciones: registrar ciudad, registrar aeropuerto, listar ciudades, listar aeropuertos por ciudad, buscar aeropuerto, renombrar, eliminar.

- **Pista 1:** para registrar un aeropuerto el usuario debe elegir una ciudad: muéstrala numerada y pide el número, no el id interno.
- **Pista 2:** muestra el `UUID` solo cuando sea necesario; para el usuario es más amigable el nombre.

Datos de ejemplo

Carga 5 ciudades y 6 aeropuertos (al menos una ciudad con dos aeropuertos). El Módulo 4 los usará para sus vuelos de prueba.

Casos de prueba que debes superar

#	Acción	Resultado esperado
1	Crear ciudad con nombre de 21 caracteres	Rechazo

#	Acción	Resultado esperado
2	Crear ciudad "Cali" y luego "cali "	La segunda es rechazada
3	Crear dos ciudades seguidas	Sus ids son consecutivos (1 y 2)
4	Crear aeropuerto sin ciudad	Rechazo
5	Crear aeropuerto con ciudad inexistente	Rechazo
6	Crear dos aeropuertos con el mismo nombre en la misma ciudad	El segundo es rechazado
7	Crear dos aeropuertos con el mismo nombre en ciudades distintas	Ambos se aceptan
8	Crear dos aeropuertos nuevos	Sus `UUID` son distintos
9	Eliminar una ciudad con aeropuertos	Rechazo con mensaje claro
10	Eliminar una ciudad sin aeropuertos	Se elimina
11	Listar aeropuertos de una ciudad	Salen ordenados por nombre

Preguntas para tu bitácora

1. ¿Por qué un `UUID` sirve como identificador sin necesidad de un contador compartido?
2. ¿Qué ventaja tienen las dos formas de construir un `Airport` (nuevo vs. existente) cuando se conecte la base de datos?
3. ¿Qué pasa con tus ids numéricos si el programa se reinicia? ¿Qué problema anticipas?
4. ¿Por qué eliminar una ciudad con aeropuertos debe estar prohibido?

Retos opcionales

- Agrega un código corto del aeropuerto (como los códigos IATA) con validación de formato.
- Agrega un método que devuelva las ciudades que no tienen ningún aeropuerto.

Plan de commits sugerido

1. `feature: agrega City y Airport` ← **primer PR**
2. `feature: agrega repositorios en memoria`
3. `feature: agrega LocationService`

4. `feature: agrega menú de ubicaciones y datos de ejemplo`

9. Módulo 3: Aerolíneas y aplicación principal

Estudiante C · Ramas `feature/modulo-3-aerolineas` y `feature/modulo-3-aplicacion` · Paquetes `com.airlinespoo.airlines` y `com.airlinespoo.app`

Tu tabla es la más pequeña (`tblAirlines`), así que construyes dos cosas: el módulo de **aerolíneas** completo (modelo, repositorio, servicio y menú) y, al final, la **aplicación principal** que ensambla los cinco módulos. Empezar por una tabla sencilla te permite dominar el recorrido completo de punta a punta, que luego repiten los demás.

Qué practicas

Clase completa con CRUD · unicidad de un campo · generador de ids con paso distinto de 1 · interfaces y polimorfismo (lista de menús) · ensamblado manual de dependencias · documentación del proyecto.

📘 **Repasa en el curso:**

- G2 · Clases y objetos, G2 · Encapsulamiento y G2 · Constructores.
- G1 · CRUD en memoria y G1 · Map y HashMap: el repositorio.
- G3 · Sobrescritura (@Override): `toString` , `equals` , `hashCode` .
- G6 · interface e implements: el `Menu` que implementa cada módulo.
- G4 · Despacho dinámico y G4 · Colecciones polimórficas: `MainMenu` recorre una lista de menús sin saber de qué módulo es cada uno.
- G0 · Maven y Git: `pom.xml` y README.

🔎 **Investiga (no está en el curso):** excepciones (`try/catch`) y `Comparator` para ordenar por nombre.

Dependencias

- **Necesitas del kit base del docente** (sección 4): excepciones, contrato del repositorio, generador de ids, validaciones, `Menu` y `ConsoleInput` .
- **Quién te necesita:** el Módulo 4 (vuelos) usa tu `Airline` . Por eso tu **primera entrega** es corta: `Airline` y su servicio.
- **La aplicación principal** depende de todos los módulos y se construye al final, a medida que cada módulo llega a `develop` .

Parte 1. Módulo de aerolíneas

Clase `Airline`

Atributo	Tipo sugerido	Regla
id	entero	Asignado por el programa: 100, 110, 120…; no cambia
name	texto	Obligatorio, máximo 50, **único** (sin distinguir mayúsculas)

- **Pista 1:** la unicidad no la puede garantizar un objeto solo. Piensa a qué clase le corresponde verificarla.
- **Pista 2:** constructor que valida con las utilidades del kit base; getters; setter del nombre con validación; `equals` / `hashCode` por id; `toString`; implementa la interfaz de identidad del kit.
- **Pista 3:** compara la numeración de aerolíneas (de 10 en 10) con la de ciudades (de 1 en 1). El generador del kit se configura con arranque y paso: ¿dónde lo configuras?

Mapa CRUD de aerolíneas

Cada método de tu repositorio y de tu servicio corresponde a una operación que más adelante será una sentencia SQL. Usa **estos nombres**, para que todos los módulos se parezcan y el cambio a JDBC o JPA sea mecánico.

Operación	En la base de datos	Repositorio	Servicio	Reglas
Crear	`INSERT` en `tblAirlines`	`create`	`createAirline` (hacía `uspCrearAerolineas`)	Nombre válido y no repetido; el id lo pide al generador
Reer uno	`SELECT … WHERE airlineId = ?`	`findById`	`getAirline`	Si no existe, excepción de "no encontrado"
Reer todos	`SELECT … ORDER BY name`	`findAll`	`listAirlines`	Ordenadas por nombre

Operación	En la base de datos	Repositorio	Servicio	Reglas
Update	`UPDATE tblAirlines SET name = ?`	`update`	`renameAirline`	Nombre válido y no repetido (puede quedar igual al propio)
Delete	`DELETE … WHERE airlineId = ?`	`deleteById`	`deleteAirline`	Por ahora se permite; en la fase de integración se impedirá si tiene vuelos
Complementario	`SELECT … WHERE name = ?`	`findByName`	`searchAirlinesByName`	Búsqueda parcial sin distinguir mayúsculas
Complementario	`SELECT COUNT(*)`	`count`	`countAirlines`	Total de aerolíneas
Complementario	`SELECT 1 … WHERE name = ?`	`existsByName`	(uso interno)	Sirve para validar unicidad

- **Pista 1:** el servicio *coordina*: pide al repositorio, comprueba reglas, lanza excepciones. No imprime nada.
- **Pista 2:** `create` del repositorio solo guarda; `update` solo actualiza un objeto que ya existe. Que no se mezclen "insertar" y "actualizar" es lo que hará fácil el paso a SQL.
- **Pista 3:** error común: validar la unicidad del nombre comparando con la propia aerolínea que se está renombrando (se "choca" consigo misma).

Repositorio `AirlineRepository`

Hereda del repositorio en memoria del kit base y agrega las búsquedas por nombre.

- **Pista 1:** ¿qué colección permite buscar por id sin recorrer todo? ¿Y por nombre? Justifica.
- **Pista 2:** los métodos de listado devuelven una copia, nunca la colección interna.

Interfaz `AirlineMenu`

Implementa el `Menu` del kit. Opciones: registrar, consultar, renombrar, listar, buscar por nombre, eliminar, volver.

- **Pista 1:** captura las excepciones del servicio y muestra mensajes en español; el menú nunca termina el programa por un dato mal digitado.
- **Pista 2:** usa `ConsoleInput` del kit para leer, no crees tu propio lector de teclado.

Datos de ejemplo

Una clase pequeña que cargue 3 aerolíneas al iniciar. Después de cargarlas, sincroniza el generador para que la siguiente aerolínea no repita id.

Parte 2. La aplicación principal (paquete `app`)

Esta parte la haces **al final**, cuando los otros módulos ya estén en `develop`. Mientras tanto puedes crear un `Main` provisional que solo arranque tu menú y agregarle los demás a medida que lleguen (haz `git pull` cada sesión).

Clase `Main` (punto de entrada)

Su trabajo es **ensamblar** el sistema, no contener lógica de negocio. En este orden:

1. Crear el lector de consola.
2. Crear los generadores de ids: aerolíneas (arranque 100, paso 10), ciudades (1, 1), vuelos (1, 1), reservas (1, 1) y detalles (1, 1).
3. Crear los repositorios de cada módulo.
4. Crear los servicios **en orden de dependencia**: clientes, ubicaciones y aerolíneas (no dependen de otros módulos) → vuelos (usa ubicaciones y aerolíneas) → reservas (usa clientes y vuelos).
5. Cargar los datos de ejemplo de cada módulo, en el mismo orden.
6. Crear los menús y entregarlos a `MainMenu`.
7. Arrancar `MainMenu`.

- **Pista 1:** dibuja en papel el grafo de dependencias entre servicios; el orden de creación sale de ahí.
- **Pista 2:** esto se llama **inyección de dependencias manual**: ningún objeto crea sus dependencias, se las entregan por el constructor. Es lo que facilitará pasar a JDBC o JPA.

Clase `MainMenu`

Menú principal con una opción por módulo (clientes, ubicaciones, aerolíneas, vuelos, reservas) y la opción de salir.

- **Pista 1:** recorre la lista de menús, muestra sus títulos como opciones y ejecuta el elegido, **sin `if` por módulo**. Eso es polimorfismo.
- **Pista 2:** captura cualquier excepción inesperada, muestra un mensaje general y vuelve al menú; un error de un módulo no debe cerrar todo el programa.

Otras responsabilidades

- **`README.md`:** qué es el proyecto, cómo compilar y ejecutar, quién hizo cada módulo y un **registro de decisiones** (por ejemplo, cómo se interpreta la fila de la silla).
- **Verificación de integración:** cada vez que el docente integre un módulo en `develop`, haz `git pull`, compila y ejecuta el programa completo; avisa si algo se rompió.
- **Recorrido de punta a punta:** escribe un guion manual que recorra todo el sistema: crear ciudad → aeropuerto → aerolínea → vuelo → cliente → reserva → manifiesto → reporte mensual.

Casos de prueba que debes superar

#	Acción	Resultado esperado
1	Registrar tres aerolíneas seguidas	Ids 100, 110 y 120
2	Registrar aerolínea con nombre repetido, también con otras mayúsculas	Rechazo
3	Registrar aerolínea con nombre vacío o de 51 caracteres	Rechazo
4	Consultar una aerolínea con un id inexistente	Mensaje "no encontrada"
5	Renombrar una aerolínea al mismo nombre que ya tenía	Se acepta
6	Renombrar una aerolínea con el nombre de otra	Rechazo
7	Listar con 3 aerolíneas	Salen ordenadas por nombre
8	Buscar por "av"	Devuelve las que contienen ese texto
9	Cargar los datos de ejemplo y registrar una aerolínea nueva	Su id no repite ninguno existente
10	Modificar la lista que devuelve el servicio	No altera el repositorio

#	Acción	Resultado esperado
11	En el menú principal, elegir una opción inexistente	Mensaje claro y vuelve al menú
12	Provocar una excepción dentro de un módulo	El programa vuelve al menú principal
13	Recorrido de punta a punta	Se completa sin errores

Preguntas para tu bitácora

1. ¿Por qué `create` y `update` son operaciones separadas en el repositorio?
2. ¿Qué ventaja da que los servicios reciban el repositorio por el constructor?
3. ¿Dónde aparece el polimorfismo en `MainMenu`? Explícalo con tus palabras.
4. ¿Por qué `Main` no contiene reglas de negocio?
5. ¿Qué cambiaría en `Main` cuando conectemos una base de datos? ¿Qué no cambiaría?

Retos opcionales

- Un mensaje de bienvenida y una pantalla "acerca de" con los integrantes del equipo.
- Una clase que imprima tablas alineadas en consola, reutilizable por todos los menús.
- Un registro simple (*log*) que escriba en un archivo los errores inesperados.

Plan de commits sugerido

Rama `feature/modulo-3-aerolineas`:

1. `feature: agrega Airline con validaciones` ← **primer PR**
2. `feature: agrega AirlineRepository en memoria`
3. `feature: agrega AirlineService con CRUD y búsqueda`
4. `feature: agrega AirlineMenu y datos de ejemplo`

Rama `feature/modulo-3-aplicacion`:

1. `feature: agrega MainMenu`
2. `feature: agrega Main que ensambla los módulos`
3. `docs: agrega README con decisiones del equipo`

10. Módulo 4: Vuelos

Estudiante D · Rama `feature/modulo-4-vuelos` · Paquete `com.airlinespoo.flights`

Tu tabla es `tblFlights`: el módulo con más reglas de negocio y el trabajo más interesante con fechas y horas. Un vuelo conecta dos aeropuertos (Módulo 2) con una aerolínea (Módulo 3), y las reservas (Módulo 5) se hacen sobre tus vuelos.

Qué practicas

Asociación entre varias clases · fechas y horas con `java.time` · interfaz `Comparable` 🔎 · mapas que agrupan listas 🔎 · clase auxiliar para el resultado de una consulta · reglas entre objetos · dependencia entre módulos a través de servicios.

📘 **Repasa en el curso:**

- G2 · Clases y objetos: un objeto que contiene otros objetos (vuelo con aerolínea y aeropuertos).
- G2 · Encapsulamiento y G2 · Constructores.
- G6 · interface e implements y G6 · Clase abstracta vs. interfaz: `Comparable` es una interfaz de Java.
- G1 · Map y HashMap y G1 · ArrayList con Map: agrupar vuelos por aerolínea.
- G1 · CRUD en memoria y G1 · Métodos y modularidad.
- G3 · Sobrescritura (@Override).

🔎 **Investiga (no está en el curso):** `LocalDate`, `LocalTime`, `LocalDateTime` y `Duration`; `Comparable` y `Comparator`; mapas que conservan el orden; los *records* de Java.

Dependencias

- **Necesitas del kit base del docente:** excepciones, contrato del repositorio, generador de ids, validaciones, `Menu` y `ConsoleInput`.
- **Necesitas del Módulo 2:** la clase `Airport` y su servicio (`LocationService`).
- **Necesitas del Módulo 3:** la clase `Airline` y su servicio (`AirlineService`).
- **Quién te necesita:** el Módulo 5 usa tu `Flight` y tu servicio.

Cómo avanzar mientras esperas a los Módulos 2 y 3

Empieza por la **lógica de fechas y horas** (duración, vuelos nocturnos, orden por salida) y por el diseño en papel de tus clases. Cuando los PR de `Airport` y `Airline` estén integrados en `develop`, haz `git pull` y continúa. **No copies `Airport` ni `Airline` a tu paquete**: tendríamos dos clases con el mismo nombre y la integración fallaría.

Clase `Flight`

Atributo	Tipo sugerido	Regla
id	entero	Asignado por el programa (1, 2, 3…); no cambia
flightName	texto	Código del vuelo, máximo 8 caracteres, único; en el negocio lo haremos obligatorio y en mayúsculas, solo letras y dígitos
departure	objeto `Airport`	Obligatorio
arrival	objeto `Airport`	Obligatorio, distinto del de salida
departureDate	`LocalDate`	Obligatoria
departureTime	`LocalTime`	Obligatoria
arrivalDate	`LocalDate`	Obligatoria
arrivalTime	`LocalTime`	Obligatoria
airline	objeto `Airline`	Obligatoria

- **Pista 1:** un vuelo *tiene* una aerolínea y dos aeropuertos. Dibuja en papel un diagrama de clases con flechas antes de programar.
- **Pista 2:** además de los getters, ofrece métodos que combinen fecha y hora (salida y llegada completas), que calculen la **duración** del vuelo y que respondan si el vuelo ya salió respecto a un momento dado. El constructor valida que la llegada sea posterior a la salida y que los aeropuertos sean distintos. 🔎 Implementa `Comparable` para ordenar por fecha y hora de salida.
- **Pista 3:** errores comunes: comparar solo las fechas e ignorar las horas (un vuelo nocturno llega al día siguiente); validar el orden de las fechas solo en el constructor y no al reprogramar; confundir `LocalTime` con `LocalDateTime`.

Clase auxiliar para el reporte

El procedimiento `uspVuelosPorAerolinea` devolvía una **fila** con aerolínea, vuelo, ciudad y aeropuerto de salida, fecha y hora de salida, ciudad y aeropuerto de llegada, fecha y hora de llegada. Crea una clase pequeña que represente esa fila, solo con datos de lectura. 🔎 Investiga qué son los *records* de Java y decide si encajan aquí.

- **Pista:** esta clase no tiene lógica de negocio; agrupa datos que viajan juntos hacia la interfaz.

Mapa CRUD de vuelos

Cada método de tu repositorio y de tu servicio equivale a una operación que luego será SQL. Usa **estos nombres**, iguales a los de los otros módulos.

Operación	En la base de datos	Repositorio	Servicio	Reglas
Crear	`INSERT` en `tblFlights`	`create`	`createFlight` (programar vuelo)	Aerolínea y ambos aeropuertos existen; código no repetido; llegada posterior a salida; aeropuertos distintos; id del generador
Reer uno	`SELECT … WHERE FlightId = ?`	`findById`	`getFlight`	Si no existe, excepción de "no encontrado"
Reer uno por código	`SELECT … WHERE FlightName = ?`	`findByFlightName`	`getFlightByName`	Si no existe, excepción de "no encontrado"
Reer todos	`SELECT … ORDER BY DateDeparture, HourDeparture`	`findAll`	`listFlights`	Ordenados por fecha y hora de salida
Update	`UPDATE tblFlights SET …`	`update`	`updateFlight` (reprogramar o cambiar aerolínea o ruta)	Mismas reglas que al crear; el código puede quedar igual al propio
Delete	`DELETE … WHERE FlightId = ?`	`deleteById`	`deleteFlight` (cancelar)	Por ahora se permite; en la fase de integración se impedirá si tiene reservas
Complementario	`SELECT … WHERE AirlineId = ?`	`findByAirline`	`listFlightsByAirline`	Ordenados por salida

Operación	En la base de datos	Repositorio	Servicio	Reglas
Complementario	`SELECT … WHERE DateDeparture BETWEEN ? AND ?`	`findByDepartureDateBetween`	`listFlightsBetween`	Rango de fechas válido (inicio no posterior al fin)
Complementario	`SELECT … WHERE Departure = ? AND Arrival = ?`	`findByRoute`	`listFlightsByRoute`	Los aeropuertos deben existir
Complementario	`SELECT COUNT(*) … WHERE AirlineId = ?`	`countByAirline`	`countFlightsByAirline`	Total de vuelos de una aerolínea
Complementario	`SELECT 1 … WHERE AirlineId = ?`	`existsByAirline`	`airlineHasFlights`	Lo usarán Aerolíneas en la fase de integración
Complementario	`SELECT 1 … WHERE Departure = ? OR Arrival = ?`	`existsByAirport`	`airportHasFlights`	Lo usará Ubicaciones en la fase de integración
Complementario	`JOIN` de la consulta del procedimiento	—	`listFlightsByAirlineInMonth` (año, mes)	Ver más abajo

- **Pista 1:** el repositorio solo accede a los datos; el **servicio** valida reglas y llama a los otros servicios.
- **Pista 2:** `create` y `update` están separados: `create` falla si el id ya existe; `update` falla si no existe.
- **Pista 3:** los métodos `exists...` te servirán más adelante para que otros módulos pregunten "¿esto tiene vuelos?" sin recorrer toda la lista.

Servicio `FlightService`

- **Pista 1:** para validar aeropuertos y aerolíneas usa el **servicio** de los Módulos 2 y 3, no su repositorio. Así tu módulo depende de una *puerta oficial* y no de los detalles internos del otro.
- **Pista 2:** recibe esos servicios por el constructor.
- **Pista 3:** error común: dejar que `FlightService` cree por sí mismo aeropuertos o aerolíneas. Él los **consulta**, no los crea.

El reporte "vuelos por aerolínea" (año y mes)

Es la traducción de `uspVuelosPorAerolinea`. Entrada: año y mes. Salida: los vuelos cuya fecha de salida cae en ese mes, **agrupados por aerolínea**, aerolíneas en orden alfabético y vuelos de cada una por fecha y hora de salida.

- **Pista 1:** valida primero la entrada (mes entre 1 y 12, año razonable). Una entrada inválida lanza una excepción, no devuelve una lista vacía.
- **Pista 2:** 🔎 una estructura natural para "grupos de vuelos por aerolínea" es un mapa cuya clave es la aerolínea y cuyo valor es una lista. Piensa qué tipo de mapa conserva el orden que necesitas.
- **Pista 3:** si ningún vuelo cumple el filtro, devuelve un resultado vacío y que la interfaz muestre "sin vuelos en ese periodo".

Repositorio e interfaz

- **`FlightRepository`:** hereda del repositorio en memoria del kit base y agrega las búsquedas del mapa CRUD. Las que filtran por fecha o aerolínea devuelven copias.
- **`FlightMenu`:** implementa el `Menu` del kit. Opciones: programar, consultar por código, listar, reprogramar o actualizar, cancelar, listar por aerolínea, listar por rango de fechas, reporte mensual.
  - Para programar un vuelo, muestra listas numeradas de aerolíneas y aeropuertos y pide el número.
  - Para leer fechas y horas usa los lectores de `ConsoleInput` e informa el formato esperado.
  - Muestra la duración en horas y minutos, no en segundos.

Datos de ejemplo

Carga 8 vuelos repartidos en al menos dos meses distintos, usando los aeropuertos del Módulo 2 y las aerolíneas del Módulo 3. Sincroniza el generador tras la carga.

Casos de prueba que debes superar

#	Acción	Resultado esperado
1	Programar vuelo con código de 9 caracteres	Rechazo
2	Programar vuelo con código repetido	Rechazo
3	Programar vuelo con salida y llegada en el mismo aeropuerto	Rechazo
4	Programar vuelo que llega antes de salir	Rechazo

#	Acción	Resultado esperado
5	Programar vuelo que sale a las 23:30 y llega a las 01:10 del día siguiente	Se acepta; duración 1 h 40 min
6	Programar vuelo con aerolínea o aeropuerto inexistente	Rechazo
7	Consultar un código de vuelo inexistente	Mensaje "no encontrado"
8	Actualizar un vuelo dejando su propio código	Se acepta
9	Actualizar un vuelo con el código de otro	Rechazo
10	Actualizar un vuelo inexistente	Excepción de "no encontrado"
11	Listar vuelos por rango de fechas con inicio posterior al fin	Rechazo
12	Listar vuelos de una aerolínea	Ordenados por fecha y hora de salida
13	Reporte de un mes sin vuelos	Resultado vacío y mensaje claro
14	Reporte con mes 13	Rechazo
15	Reporte de un mes con vuelos de 2 aerolíneas	Dos grupos, aerolíneas en orden alfabético
16	`airlineHasFlights` con una aerolínea que tiene vuelos / sin vuelos	Verdadero / falso

Preguntas para tu bitácora

1. ¿Por qué un vuelo guarda objetos `Airport` y `Airline` y no sus identificadores?
2. ¿Qué diferencia hay entre `LocalTime` y `LocalDateTime` y por qué importa para vuelos nocturnos?
3. ¿Qué significa implementar `Comparable` y cómo lo usa una lista al ordenarse?
4. ¿Por qué `FlightService` usa los servicios de los otros módulos y no sus repositorios?
5. ¿Con qué sentencias SQL se correspondería cada método de tu mapa CRUD? Escríbelo con tus palabras.

Retos opcionales

- Una enumeración de estado de vuelo (programado, en vuelo, finalizado) calculada con la fecha y hora actuales.
- Un reporte de la aerolínea con más vuelos en un mes.
- Agregar una capacidad máxima de pasajeros por vuelo *(no existe en el modelo: discútelo con el docente, porque afecta al Módulo 5)*.

Plan de commits sugerido

1. `feature: agrega Flight con validaciones de fechas`
2. `feature: agrega FlightRepository en memoria`
3. `feature: agrega FlightService con CRUD`
4. `feature: agrega consultas complementarias y existencias`
5. `feature: agrega reporte de vuelos por aerolínea y mes`
6. `feature: agrega FlightMenu y datos de ejemplo`

11. Módulo 5: Reservas y pasajeros

Estudiante E · Rama `feature/modulo-5-reservas` · Paquete `com.airlinespoo.reservations`

Construyes el corazón del negocio: un cliente reserva un vuelo y registra a uno o varios pasajeros con su silla. Es el módulo que **más depende de los demás**, así que planifica bien tus primeras sesiones.

Qué practicas

Composición (una reserva *contiene* sus detalles) · clases pequeñas que representan un valor (la silla) 🔎 · copias defensivas de colecciones · reglas que cruzan varios objetos y varios módulos · listas, conjuntos y mapas · coordinación entre servicios.

📘 **Repasa en el curso:**

- G2 · Encapsulamiento: no exponer la lista interna y devolver copias.
- G2 · Clases y objetos y G2 · Constructores.
- G1 · List y ArrayList, G1 · Map y HashMap y G1 · CRUD en memoria.
- G3 · Sobrescritura (@Override): `equals` y `hashCode` de `Seat`.

🔎 **Investiga (no está en el curso):** `Set` y `HashSet`, listas de solo lectura, el concepto de composición y los objetos inmutables.

Dependencias

- **Necesitas del kit base del docente:** excepciones, contrato del repositorio, generador de ids.
- **Necesitas del Módulo 1:** `Customer` y `CustomerService`.
- **Necesitas del Módulo 4:** `Flight` y `FlightService`.
- **Quién te necesita:** nadie directamente; tu módulo es el que cierra el flujo completo del sistema.

Qué puedes avanzar sin esperar a nadie

1. La clase de la silla y la de detalle de reserva (`ReservationDetail`): solo dependen de tipos básicos de Java.
2. Las reglas de pasajeros dentro de la reserva, usando temporalmente datos de prueba.
3. El diseño en papel del servicio y del menú.

Cuando los Módulos 1 y 4 estén integrados en `develop`, haz `git pull` y completa `Reservation` y los servicios.

Clase `Seat` (silla) 🔎

La base de datos guarda fila y número de silla como dos columnas sueltas. En POO es mejor agruparlas en un objeto pequeño que **representa un valor**: la silla "12B" es la misma silla sin importar cuántas veces la crees.

Atributo	Tipo sugerido	Regla
rowNumber	texto de máximo 2 caracteres	Obligatorio, en mayúsculas
seatNumber	numérico pequeño	Entre 1 y un máximo que defina el equipo (por ejemplo 99)

- **Pista 1:** dos sillas son iguales cuando fila y número coinciden. ¿Qué método debes redefinir para que Java lo sepa?
- **Pista 2:** objeto **inmutable**: sin setters; si el pasajero cambia de silla, se reemplaza la silla completa.

- **Pista 3:** documenta en tu README cómo interpretas "fila" (¿solo números? ¿letra y número?) para que el equipo y el docente conozcan tu decisión.

Clase `ReservationDetail`

Atributo	Tipo sugerido	Regla
id	entero	Asignado por el programa; no cambia
passengerFullName	texto	Obligatorio, máximo 50
passengerDocument	texto	Obligatorio, máximo 20
seat	objeto `Seat`	Obligatoria

- **Pista 1:** un detalle describe **un pasajero con su silla** dentro de una reserva. No guarda el vuelo: eso lo sabe la reserva.
- **Pista 2:** constructor que valida todo; getters; `equals` / `hashCode` por id; `toString` legible.
- **Pista 3:** error común: permitir un documento con espacios o con formato libre. Define una regla (por ejemplo, quitar espacios y exigir letras o dígitos) y úsala siempre.

Clase `Reservation`

Atributo	Tipo sugerido	Regla
id	entero	Asignado por el programa; no cambia
customer	objeto `Customer`	Obligatorio
flight	objeto `Flight`	Obligatorio
details	lista de `ReservationDetail`	Al menos 1 pasajero, máximo el que defina el equipo (por ejemplo 9)

- **Pista 1:** esto es **composición**: si se elimina la reserva, sus detalles dejan de tener sentido. Una `ReservationDetail` nunca vive sola.
- **Pista 2:** la reserva controla su lista: ofrece métodos para *agregar* y *quitar* pasajeros y para consultar la lista **sin dejar que de afuera la modifiquen**. 🔎 Devuelve una copia o una vista de solo lectura.

- **Pista 3:** reglas internas de la reserva: no repetir el mismo documento de pasajero, no repetir la misma silla, no superar el máximo, no quedar sin pasajeros (decide qué ocurre al quitar el último).

Repositorios

Uno para reservas (los detalles se guardan **dentro** de la reserva, no en un repositorio aparte).

- **Pista 1:** ¿por qué los detalles no necesitan su propio repositorio? Escríbelo en tu bitácora y relaciónalo con la composición.
- **Pista 2:** el repositorio debe permitir buscar reservas por cliente y por vuelo.

Servicio `ReservationService`

Operación	Reglas que debe cumplir
Crear reserva	El cliente y el vuelo deben existir; el vuelo no puede haber salido ya; debe incluir al menos un pasajero
Agregar pasajero	La silla no puede estar ocupada **por otro pasajero en ese mismo vuelo** (revisa todas las reservas del vuelo)
Quitar pasajero	No puede dejar la reserva vacía
Cambiar silla	La nueva silla debe estar libre en ese vuelo
Cancelar reserva	Se elimina la reserva con todos sus detalles
Consultar reservas de un cliente	Ordenadas por fecha de salida del vuelo
Listar sillas ocupadas de un vuelo	Ordenadas por fila y número
Lista de pasajeros de un vuelo (manifiesto)	Nombre, documento y silla, ordenado por silla
Contar pasajeros de un vuelo	Total entre todas sus reservas

- **Pista 1:** la regla de la silla es *la más importante y la más difícil* del sistema: no puede validarla un solo objeto, pues depende de **todas** las reservas del mismo vuelo. Dibuja el escenario en papel con dos reservas y una silla.

- **Pista 2:** usa los servicios de los Módulos 1 y 4 para obtener el cliente y el vuelo; recíbelos por el constructor.
- **Pista 3:** 🔎 un `Set` (conjunto) es ideal para responder "¿esta silla ya está ocupada?". Piensa por qué necesitas `equals` y `hashCode` bien hechos en `Seat`.
- **Pista 4:** error común: validar la silla solo dentro de la misma reserva y olvidar las demás.

Mapa CRUD de reservas y pasajeros

En la base de datos son **dos tablas** (`tblReservations` y `tblDetailsReservation`); en Java la `Reservation` es la dueña de sus detalles (composición), así que las operaciones sobre pasajeros se hacen a través de ella. Usa **estos nombres**, iguales a los de los otros módulos.

Operación	En la base de datos	Repositorio	Servicio	Reglas
Crear reserva	`INSERT` en `tblReservations` y un `INSERT` en `tblDetailsReservation` por pasajero	`create`	`createReservation`	Cliente y vuelo existen; vuelo no salido; al menos un pasajero; sillas libres en ese vuelo
Crear pasajero	`INSERT` en `tblDetailsReservation`	`update` (de la reserva)	`addPassenger`	Silla libre en el vuelo; documento no repetido en la reserva; no superar el máximo
Reer una reserva	`SELECT` con `JOIN` de reserva y detalles	`findById`	`getReservation`	Si no existe, excepción de "no encontrada"
Reer todas	`SELECT` de reservas	`findAll`	`listReservations`	Ordenadas por id
Reer pasajeros	`SELECT … WHERE ReservationId = ?`	`findById`	`listPassengers`	Reserva existente
Update reserva	`UPDATE tblReservations SET FlightId = ?`	`update`	`changeReservationFlight`	Nuevo vuelo existente y no salido; las sillas deben estar libres en el nuevo vuelo
Update pasajero	`UPDATE tblDetailsReservation SET …`	`update` (de la reserva)	`changeSeat` , `updatePassenger`	Silla libre; datos válidos
Delete pasajero	`DELETE` en `tblDetailsReservation`	`update` (de la reserva)	`removePassenger`	No puede dejar la reserva vacía
Delete reserva	`DELETE` de detalles y luego de la reserva	`deleteById`	`cancelReservation`	Elimina la reserva con todos sus detalles
Complementario	`SELECT … WHERE CustomerId = ?`	`findByCustomer`	`listReservationsByCustomer`	Ordenadas por fecha de salida del vuelo
Complementario	`SELECT … WHERE FlightId = ?`	`findByFlight`	`listReservationsByFlight`	Vuelo existente

Operación	En la base de datos	Repositorio	Servicio	Reglas
Complementario	`SELECT 1 … WHERE CustomerId = ?`	`existsByCustomer`	`customerHasReservations`	Lo usará Clientes en la fase de integración
Complementario	`SELECT 1 … WHERE FlightId = ?`	`existsByFlight`	`flightHasReservations`	Lo usará Vuelos en la fase de integración
Complementario	`SELECT RowNumber, SeatNumer … WHERE FlightId = ?`	(sobre `findByFlight`)	`listOccupiedSeats`	Ordenadas por fila y número
Complementario	`SELECT 1 … WHERE FlightId = ? AND RowNumber = ? AND SeatNumer = ?`	(sobre `findByFlight`)	`isSeatOccupied`	Es la regla central de las sillas
Complementario	`SELECT COUNT(*)` con `JOIN`	(sobre `findByFlight`)	`countPassengersByFlight`	Total entre todas las reservas del vuelo
Complementario	`JOIN` de detalles, reservas y clientes	(sobre `findByFlight`)	`getFlightManifest`	Nombre, documento y silla, ordenado por silla

- **Pista 1:** que las operaciones sobre pasajeros pasen por `update` de la reserva es una consecuencia de la composición. Escribe en tu bitácora cómo se vería eso en SQL (varias tablas en una misma operación).
- **Pista 2:** `create` falla si el id de la reserva ya existe; `update` falla si no existe.
- **Pista 3:** los métodos `exists…` permitirán que otros módulos pregunten "¿esto tiene reservas?" sin recorrer todas las reservas.

Interfaz `ReservationMenu`

Opciones: crear reserva, agregar pasajero, quitar pasajero, cambiar silla, cancelar reserva, consultar por cliente, ver sillas ocupadas, ver manifiesto del vuelo.

- **Pista 1:** para elegir cliente y vuelo, pide el documento y el código del vuelo, y confirma mostrando el nombre y la ruta antes de continuar.
- **Pista 2:** crear una reserva es un **flujo en pasos** (cliente → vuelo → pasajeros). Si algo falla a mitad de camino, decide qué ocurre con lo ya capturado.
- **Pista 3:** muestra el manifiesto como una tabla alineada en consola.

Datos de ejemplo

Carga 4 reservas de muestra que usen clientes del Módulo 1 y vuelos del Módulo 4, con 1 a 3 pasajeros cada una y sin sillas repetidas en un mismo vuelo.

Casos de prueba que debes superar

#	Acción	Resultado esperado
1	Crear reserva sin pasajeros	Rechazo
2	Crear reserva con cliente inexistente	Rechazo
3	Crear reserva con vuelo inexistente	Rechazo
4	Crear reserva en un vuelo que ya salió	Rechazo
5	Agregar dos pasajeros con el mismo documento en una reserva	El segundo es rechazado
6	Asignar la silla 12B a dos pasajeros de la misma reserva	Rechazo
7	Asignar la silla 12B a un pasajero de otra reserva del mismo vuelo	Rechazo
8	Asignar la silla 12B en dos vuelos distintos	Se acepta
9	Agregar más pasajeros del máximo permitido	Rechazo
10	Quitar el único pasajero	Rechazo
11	Cancelar una reserva y volver a usar su silla	La silla queda libre
12	Ver manifiesto con 5 pasajeros	Salen ordenados por silla
13	Intentar modificar la lista que devuelve la reserva desde afuera	No afecta la reserva

Preguntas para tu bitácora

1. ¿Qué diferencia hay entre composición y asociación? Da un ejemplo de cada una en este sistema.
2. ¿Por qué es mejor que `Seat` sea inmutable?
3. ¿Qué problema aparece si `Reservation` devuelve su lista interna de detalles?
4. ¿Por qué la regla de "silla ocupada" se valida en el servicio y no en la reserva?
5. ¿Qué pasaría con las reservas si se elimina un vuelo? ¿Cómo lo impide la base de datos con su clave foránea?

Retos opcionales

- Un cálculo de ocupación del vuelo (porcentaje de sillas usadas respecto de una capacidad dada).
- Impedir que un cliente tenga dos reservas en vuelos que se solapan en fecha y hora.

Plan de commits sugerido

1. `feature: agrega Seat y ReservationDetail`
2. `feature: agrega Reservation con composición de detalles`
3. `feature: agrega repositorio en memoria`
4. `feature: agrega ReservationService con regla de sillas`
5. `feature: agrega menú, manifiesto y datos de ejemplo`

12. Orden de integración, dependencias y calendario

Los módulos no se integran en cualquier orden: el kit base va primero porque todos lo usan, Ubicaciones y Aerolíneas van antes que Vuelos, y Reservas va casi al final porque usa a Clientes y a Vuelos. Esta sección es el mapa de cómo se juntan las piezas en `develop`.

Quién depende de quién

Módulo	Tablas	Necesita de	Lo necesitan
1 · Clientes y catálogos	`tblCountries`, `tblDocumentsType`, `tblCustomers`	Kit base	5
2 · Ubicaciones	`tblCities`, `tblAirports`	Kit base	4
3 · Aerolíneas y aplicación	`tblAirlines`	Kit base (la aplicación necesita a todos)	4
4 · Vuelos	`tblFlights`	Kit base, 2 y 3	5
5 · Reservas	`tblReservations`, `tblDetailsReservation`	Kit base, 1 y 4	Nadie

Regla práctica: nunca copies una clase de otro módulo para "salir del paso". Espera su Pull Request integrado en `develop`, haz `git pull` y úsala. Mientras tanto, avanza en las partes que no dependen de nadie (cada módulo trae una lista de qué puedes adelantar).

Orden de integración en `develop`

Orden	Pull Request	Por qué en este lugar
0	Kit base del docente	Todos necesitan excepciones, contrato del repositorio, generador y validaciones
1	Módulo 2 · `City` y `Airport` (entrega anticipada)	Desbloquea al Módulo 4
2	Módulo 3 · `Airline` y su servicio (entrega anticipada)	Desbloquea al Módulo 4
3	Módulo 1 · Clientes y catálogos	Desbloquea al Módulo 5
4	Módulo 2 · Repositorios, servicio y menú	Completa ubicaciones
5	Módulo 3 · Repositorio, menú y datos de ejemplo	Completa aerolíneas
6	Módulo 4 · Vuelos	Necesita ubicaciones y aerolíneas ya integradas
7	Módulo 5 · Reservas	Necesita clientes y vuelos ya integrados
8	Módulo 3 · Aplicación (`feature/modulo-3-aplicacion`)	Ensambla todo

Si un PR llega fuera de orden, el docente puede pedir que se espere. Mientras esperas, no te quedes quieto: mejora tus casos de prueba, tu bitácora y tus comentarios.

Calendario sugerido por sesiones

Ajusta el número de sesiones a tu curso; lo importante es el orden.

Sesión	Objetivo	Entregable en `develop`
0	El docente publica el kit base, el repositorio, las ramas, las protecciones y los Issues; cada estudiante clona, configura Git y compila	Repositorio y kit base listos
1	Lectura guiada del kit base y modelos sin dependencias	PR anticipados de los Módulos 2 y 3
2	Modelos restantes (clientes y catálogos, vuelo, silla y detalle)	PR del Módulo 1 (modelos)
3	Repositorios y servicios con su mapa CRUD	PR de los Módulos 1, 2 y 3 completos
4	Menús, datos de ejemplo y reglas de vuelos	PR del Módulo 4
5	Reservas y regla de sillas	PR del Módulo 5
6	Aplicación principal, recorrido de punta a punta y reglas cruzadas (abajo)	PR de la aplicación (Módulo 3)
7	Revisión final, defensa individual y publicación de la versión 1.0.0	`release/1.0.0` → `main` con etiqueta

Fase de integración: reglas que cruzan módulos

Durante los módulos dejaste pendientes unas reglas de eliminación porque dependían de otros módulos. Ahora que todo está en `develop`, el equipo las resuelve **en una sesión guiada por el docente**. Cada módulo ya dejó listos los métodos de consulta (`exists…` y `…Has…`) en su mapa CRUD:

Regla	La hace cumplir	Necesita preguntar a
No eliminar una aerolínea con vuelos	Aerolíneas (Módulo 3)	Vuelos (Módulo 4): `airlineHasFlights`
No eliminar un aeropuerto con vuelos	Ubicaciones (Módulo 2)	Vuelos (Módulo 4): `airportHasFlights`
No eliminar un vuelo con reservas	Vuelos (Módulo 4)	Reservas (Módulo 5): `flightHasReservations`

Regla	La hace cumplir	Necesita preguntar a
No eliminar un cliente con reservas	Clientes (Módulo 1)	Reservas (Módulo 5): `customerHasReservations`

El problema de diseño: Vuelos ya depende de Ubicaciones y Aerolíneas, y Reservas ya depende de Clientes y Vuelos. Si ahora Aerolíneas, Ubicaciones, Vuelos y Clientes necesitan preguntar "hacia abajo", queda una **dependencia circular** (A necesita a B y B necesita a A) y el programa no se puede armar de forma ordenada. Es una situación real y frecuente.

Preguntas para discutir en equipo, antes de escribir código:

1. ¿Qué módulos terminan dependiendo de cuáles en cada regla? Dibuja las flechas.
2. ¿Qué tipo de contrato (interfaz) podría definir el **kit base** para que "el que borra" pregunte "¿esto está en uso?" sin conocer al módulo que responde?
3. ¿Quién registra a los "que responden" y en qué lugar del programa se hace? Pista: piensa en `Main`.
4. ¿Qué cambia respecto a la clave foránea de la base de datos, que resuelve esto automáticamente?

El docente guía la solución final; el objetivo es que entiendas el problema, no que lo resuelvas solo.

Qué hace el docente al integrar cada PR

1. Revisa el código y los comentarios del PR (y la revisión entre pares).
2. Descarga la rama, compila y ejecuta los casos de prueba del módulo.
3. Integra en `develop` con un commit de fusión.
4. Avisa al equipo para que haga `git pull` en `develop` y actualice sus ramas.
5. Cuando termina el ejercicio, publica la versión 1.0.0 siguiendo el flujo `release/*` de G0 · Gitflow.

13. Checklist de entrega y criterios de evaluación

Antes de abrir tu Pull Request recorre esta lista completa. Un PR que no la cumple vuelve a tu rama sin revisión.

Checklist de entrega de tu módulo

Diseño y POO

- [ ] Todas las clases del módulo existen y cada una está en su propio archivo.
- [ ] Todos los atributos son privados.
- [ ] Los constructores validan y dejan el objeto en estado válido.
- [ ] Los identificadores no tienen setter.
- [ ] Las relaciones entre clases son referencias a objetos, no ids sueltos.
- [ ] `toString`, `equals` y `hashCode` están sobrescritos donde corresponde.
- [ ] Ninguna colección interna se expone directamente.

Reglas y calidad

- [ ] Todas las reglas de la tabla de tu módulo están implementadas.
- [ ] Los errores se comunican con las excepciones del núcleo, con mensajes en español.
- [ ] Ninguna clase de `model`, `repository` o `service` imprime o lee de la consola.
- [ ] No hay números ni textos "mágicos" repetidos; hay constantes.
- [ ] Los métodos son cortos y hacen una sola cosa.
- [ ] Hay Javadoc en las clases y métodos públicos.

Pruebas

- [ ] Ejecuté a mano **todos** los casos de prueba de mi módulo y los marqué como superados.
- [ ] Cargué los datos de ejemplo y el programa arranca sin errores.

Git y entrega

- [ ] Mi rama es `feature/modulo-N-tema` y nació de `develop`.
- [ ] Los commits son pequeños y siguen el formato `tipo: descripción`.
- [ ] Traje lo último de `develop` antes de abrir el PR.
- [ ] `git status` no muestra archivos ajenos a mi módulo ni archivos generados.
- [ ] El PR apunta a `develop`, tiene título, descripción y revisor.

Estudio

- [ ] Mi bitácora está al día, con las preguntas de mi módulo respondidas con mis palabras.
- [ ] Puedo explicar en voz alta cualquier clase de mi módulo.

Criterios de evaluación (propuesta que el docente puede ajustar)

Criterio	Peso	Qué se observa
Modelado y POO	25 %	Encapsulamiento, constructores, relaciones entre objetos, `equals` / `hashCode` / `toString`, uso correcto de herencia o interfaces donde aplican
Reglas y validaciones	15 %	Se cumplen las reglas de la guía; excepciones adecuadas; mensajes claros
Repositorio y servicio	15 %	Separación de responsabilidades, uso de colecciones adecuadas, dependencias recibidas por constructor
Interfaz de consola	5 %	El menú es usable y no se rompe con datos inválidos
Casos de prueba	10 %	Los casos de la guía fueron ejecutados y los resultados coinciden
Git y Pull Request	10 %	Rama correcta, commits claros, PR completo, revisión de un compañero
Bitácora	5 %	Intentos, errores y aprendizajes honestos; preguntas respondidas
Defensa individual	15 %	Explica su código, modifica algo pequeño en vivo y justifica decisiones

La defensa individual

El docente puede pedirte, con tu código abierto:

- Explicar por qué un atributo es privado o un identificador no tiene setter.
- Decir qué ocurre al ejecutar un caso de prueba concreto y en qué línea se detecta el error.
- Cambiar una regla (por ejemplo, el largo máximo de un campo o la lista de formatos válidos) y mostrar que funciona.
- Explicar qué parte de tu módulo se vería afectada si cambiara el modelo de datos.
- Contar qué te costó más y qué harías distinto.

La defensa es oral y breve. No se evalúa la perfección, sino que entiendas lo que entregaste.

Bitácora personal: formato sugerido

Fecha	Qué intenté	Qué pasó (error, resultado)	Qué aprendí	Ayuda que usé (nivel 1 a 6, sección 2)

Si usaste IA en algún momento, anótalo en la última columna y describe en una frase qué le pediste. Hacerlo no se penaliza; ocultarlo, sí.

14. Puente hacia JDBC y JPA

Cuando conectemos una base de datos, **tus clases del modelo y tus servicios casi no cambiarán**; lo que se reemplaza es la capa de repositorios. Si diseñaste bien, el cambio será pequeño. Si dejaste lógica de negocio dentro de los repositorios o impresiones dentro de los servicios, ahí dolerá.

📘 **Lecciones del curso que vienen después:**

- G7 · JDBC directo: conexión a SQL Server
- G7 · JDBC: CRUD con SQL directo y procedimientos almacenados
- G7 · JPA y Hibernate
- G7 · CRUD con EntityManager
- G7 · Relaciones: @ManyToOne

Qué cambia y qué se queda

Pieza de tu proyecto	¿Cambia?	Qué ocurrirá
Clases del modelo	Poco	Se les indicará a qué tabla y columna corresponde cada atributo
Interfaz `Repository`	No	Es el contrato; sigue igual
Repositorios en memoria	Se reemplazan	Aparecerán versiones con JDBC o JPA que cumplen el mismo contrato
Servicios	Casi no	Reciben el nuevo repositorio por el constructor

Pieza de tu proyecto	¿Cambia?	Qué ocurrirá
Menús y `ConsoleInput`	No	No saben de dónde vienen los datos
`Main`	Sí	Elige qué repositorios crear (memoria, JDBC o JPA)
`IdGenerator`	En parte	Las tablas con identidad generan ids; las que no (reservas y detalles) seguirán necesitando un generador
Datos de ejemplo	Sí	Pasan a ser filas en la base de datos

De tus clases a las tablas

La base de datos está definida en los scripts de SQL Server y PostgreSQL del proyecto. Así se corresponden:

Clase	Tabla	Detalle a recordar
`Country`	`tblCountries`	El id no es autogenerado
`DocumentType`	`tblDocumentsType`	El id es un texto de 2 caracteres
`Customer`	`tblCustomers`	`firstName` ↔ columna `FirtName`; el correo es único
`City`	`tblCities`	El id lo genera el motor (empieza en 1, de 1 en 1)
`Airport`	`tblAirports`	El id es un `UUID`; la ciudad es la clave foránea `CityId`
`Airline`	`tblAirlines`	El id lo genera el motor (empieza en 100, de 10 en 10); el nombre es único
`Flight`	`tblFlights`	Dos claves foráneas a aeropuertos (`Departure`, `Arrival`); el código es único
`Reservation`	`tblReservations`	El id **no** es autogenerado; claves foráneas a cliente y vuelo
`ReservationDetail` y `Seat`	`tblDetailsReservation`	`Seat` se reparte en dos columnas: `RowNumber` y `SeatNumer`

Lo que ya entrenaste sin saberlo

- **Clave foránea → referencia a objeto.** Tu `Flight` guarda un `Airline` y dos `Airport`: es exactamente lo que hará `@ManyToOne` en JPA.
- **Composición → cascada.** La reserva "posee" sus detalles; en la base de datos eso se ve como una clave foránea del detalle hacia la reserva.
- **Restricción única → regla de unicidad.** El motor también la hará cumplir, pero tu validación en el servicio da un mensaje amable antes de llegar al error de la base.
- **Procedimientos almacenados → métodos de servicio.** `uspCrearClientes`, `uspCrearAerolineas` y `uspVuelosPorAerolinea` ya existen en tu código; más adelante veremos cómo llamar a los originales desde JDBC.
- **Tipos.** `LocalDate`, `LocalTime` y `UUID` tienen correspondencia directa con `DATE`, `TIME` y el identificador único de la base.

Qué debes dejar listo hoy para que la transición sea fácil

- [ ] Los servicios reciben sus repositorios por el constructor (no los crean).
- [ ] Los servicios dependen de la **interfaz** `Repository`, no de la clase en memoria.
- [ ] Ninguna regla de negocio vive dentro de un repositorio.
- [ ] Los ids no se modifican y cada entidad implementa `Identifiable`.
- [ ] Los constructores distinguen "objeto nuevo" de "objeto que viene de la base" (al menos en `Airport`).
- [ ] Los menús no imprimen detalles técnicos (como el `UUID`) salvo que sea necesario.
- [ ] Tus decisiones están anotadas en el README.

Preguntas para pensar antes de la próxima clase

1. ¿Qué pasará con los datos de ejemplo cuando el programa se cierre, ahora que usan memoria?
2. Si el motor genera el id de una ciudad, ¿cuándo conoce tu programa ese id: antes o después de guardar?
3. ¿Qué sucede en tu `ReservationService` si el vuelo se elimina desde otra conexión mientras creas la reserva?
4. ¿Cómo comprobarías que la implementación JDBC se comporta igual que la de memoria con tus mismos casos de prueba?

15. Glosario y preguntas de autoevaluación

Glosario

Término	Significado en este proyecto	Lección del curso
Clase	Plantilla que describe los datos y el comportamiento de un tipo de objeto (`Customer`, `Flight`)	G2 · Clases y objetos
Objeto	Instancia concreta de una clase en memoria	G2 · Clases y objetos
Encapsulamiento	Atributos privados y acceso controlado mediante métodos	G2 · Encapsulamiento

Término	Significado en este proyecto	Lección del curso
Constructor	Método especial que crea el objeto y lo deja en estado válido	G2 · Constructores
Sobrescritura	Redefinir en una clase un método heredado (`toString` , `equals` , `hashCode`)	G3 · Sobrescritura
Herencia	Una clase reutiliza y extiende a otra (las excepciones del kit base)	G3 · extends y super
Polimorfismo	Un mismo mensaje con comportamientos distintos según el objeto (la lista de menús)	G4 · Despacho dinámico
Clase abstracta	Plantilla incompleta a propósito que no se instancia (`InMemoryRepository`)	G5 · Clase abstracta
Interfaz	Contrato de comportamiento que otras clases cumplen (`Repository` , `Menu`)	G6 · interface e implements
Asociación	Un objeto guarda una referencia a otro (un vuelo guarda su aerolínea)	G2 · Clases y objetos
Composición	Un objeto *posee* a otros que no existen sin él (una reserva y sus detalles)	🔎
Colección	Estructura que agrupa objetos (`List` , `Map`)	G1 · List y ArrayList · G1 · Map y HashMap
CRUD	Las cuatro operaciones sobre datos: Create, Read, Update, Delete (`INSERT` , `SELECT` , `UPDATE` , `DELETE`)	G1 · CRUD en memoria
Repositorio	Objeto que oculta cómo se guardan los datos y ofrece operaciones CRUD	G1 · CRUD en memoria
Servicio	Objeto que aplica las reglas de negocio usando repositorios y otros servicios	G1 · Métodos y modularidad

Término	Significado en este proyecto	Lección del curso
Inyección de dependencias	Entregar a un objeto, por su constructor, lo que necesita en lugar de que lo cree él	🔎
Excepción	Objeto que describe un error y interrumpe el flujo normal	🔎
Integridad referencial	Regla que impide referenciar algo que no existe o eliminar algo en uso (claves foráneas)	G7 · Relaciones: @ManyToOne
Rama (`feature/*`)	Línea de trabajo para una funcionalidad, que nace de `develop` y vuelve a `develop`	G0 · Gitflow
Pull Request	Solicitud de revisión e integración de tu rama	G0 · Trabajo colaborativo

Preguntas de autoevaluación

Responde primero sin mirar tus apuntes; luego verifica. Si no puedes responder una, vuelve a la lección indicada y a tu código.

Clases y encapsulamiento

1. ¿Qué pasa si un atributo es público y alguien lo modifica sin pasar por el constructor?
2. ¿Por qué un identificador no debe tener setter?
3. ¿Qué diferencia hay entre validar en el constructor y validar en el servicio? Da un ejemplo de cada caso.

Relaciones entre objetos

4. ¿Qué diferencia hay entre asociación y composición? Da un ejemplo de cada una en este sistema.
5. ¿Por qué un `Flight` guarda un `Airline` y no solo su identificador?
6. ¿Qué problema aparece si una clase devuelve su lista interna?

Herencia, polimorfismo e interfaces

7. ¿Para qué sirve que todas las excepciones del sistema hereden de una base común?
8. ¿Dónde aparece el polimorfismo en `MainMenu`?

9. ¿Cuándo usarías una interfaz y cuándo una clase abstracta? Usa `Repository` e `InMemoryRepository` como ejemplo.

Colecciones

10. ¿Cuándo es mejor un mapa que una lista para un repositorio?
11. ¿Qué ocurre con un `HashMap` si `equals` y `hashCode` de la clave no están bien definidos?

CRUD y capas

12. Escribe, con tus palabras, qué sentencia SQL equivale a cada método del repositorio de tu módulo.
13. ¿Por qué `create` y `update` son operaciones separadas?
14. ¿Por qué el repositorio no lanza la excepción de "no encontrado"?
15. Dibuja las cuatro capas del proyecto y las flechas de dependencia entre ellas.

Reglas entre módulos

16. ¿Por qué la regla de "silla ocupada" no puede validarla un solo objeto?
17. ¿Qué es una dependencia circular y en qué regla de eliminación aparece?

Git y trabajo en equipo

18. ¿Desde qué rama nace tu rama `feature/*` y hacia cuál se integra?
19. ¿Qué haces si al abrir tu Pull Request aparece un conflicto en un archivo que no es de tu módulo?
20. ¿Por qué los commits pequeños con mensajes claros ayudan al equipo?

Cierre

Si llegaste hasta aquí con tu módulo funcionando, tu bitácora al día y tu Pull Request integrado, ya tienes lo necesario para dar el siguiente paso: reemplazar la capa de repositorios en memoria por una base de datos con JDBC o JPA.